

beamer named overlay specifications with **beanoves**

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is another solution to this [latex.org forum query](#).

Contents

1 Implementation

1.1 Package declarations

```
1 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
2 \ProvidesExplPackage
3   {beanoves}
4   {2025/06/03}
5   {1.0}
6   {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string **beanoves** or containing the case insensitive string **bnvs** delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into **beamer** specifications based on this data model.

The development is made easier due to a facility layer over **expl**.

1.3 Facility layer: definitions and naming

In order to make the code shorter and easier to read during development, we add a layer over $\text{\LaTeX}3$. The `c` and `v` argument specifiers take a slightly different meaning when used in a function which name contains `bnvs` or `BNVS`. Where $\text{\LaTeX}3$ would transform `l__bnvs_ref_tl` into `\l__bnvs_ref_tl`, `bnvs` will directly transform `ref` into `\l__bnvs_ref_tl`. The type of the local variable used depends on the context and may be `seq` or `int` for example. There are however a pair of exceptions mentioned below. For a better reading experience, “`ref`” will generally stand for `\l__bnvs_ref_tl`, whereas “`path sequence`” will generally stand for `\l__bnvs_path_seq`. Other similar shortcuts are used as well.

Functions with `BNVS` in their names are management functions. They belong to a deeper layer and do not really contain any logic specific to the `beanoves` package.

<code>\BNVS:c</code>	<code>\BNVS:c {<cs core name>}</code>
<code>\BNVS_l:cn</code>	<code>\BNVS_l:cn {<local variable core name>} {<type>}</code>
<code>\BNVS_g:cn</code>	<code>\BNVS_g:cn {<global variable core name>} {<type>}</code>

These are naming functions internally used to focus on the discriminating part of variable or function names.

```

7 \cs_new:Npn \BNVS:c #1 { __bnvs_#1 }
8 \cs_new:Npn \BNVS_l:cn #1 #2 { l__bnvs_#1_#2 }
9 \cs_new:Npn \BNVS_g:cn #1 #2 { g__bnvs_#1_#2 }
```

<code>\BNVS_use_Raw:N</code>	<code>\BNVS_use_Raw:N <cs></code>
<code>\BNVS_use_Raw:c</code>	<code>\BNVS_use_Raw:c {<cs name>}</code>
<code>\BNVS_use_Raw:Nc</code>	<code>\BNVS_use_Raw:Nc <function> {<cs name>}</code>
<code>\BNVS_use_Raw:nc</code>	<code>\BNVS_use_Raw:nc {<tokens>} {<cs name>}</code>
<code>\BNVS_use:c</code>	<code>\BNVS_use:c {<cs core>}</code>
<code>\BNVS_use:Nc</code>	<code>\BNVS_use:Nc <function> {<cs core>}</code>
<code>\BNVS_use:nc</code>	<code>\BNVS_use:nc {<tokens>} {<cs core>}</code>

`\BNVS_use_Raw:c` is a convenient wrapper over `\use:c`. possibly prepended with some code, for debugging and testing. It needs 3 expansion steps just like `\BNVS_use:c`. The other are used to expand `\use:c` enough before usage by `<function>` or `<tokens>`. The first argument of `<function>` has type `N`. The next token after `<tokens>` will have type `N` too. `<cs name>` is a full cs name whereas `<cs core>` will be prepended with the appropriate prefix specific to the `beanoves` package.

```

10 \cs_new:Npn \BNVS_use_Raw:N #1 { #1 }
11 \cs_new:Npn \BNVS_use_Raw:c #1 {
12   \exp_last_unbraced:No
13   \BNVS_use_Raw:N { \cs:w #1 \cs_end: }
14 }
15 \cs_new:Npn \BNVS_use:c #1 {
16   \BNVS_use_Raw:c { \BNVS:c { #1 } }
17 }
18 \cs_new:Npn \BNVS_use_Raw:NN #1 #2 {
19   #1 #2
20 }
```

```

21 \cs_new:Npn \BNVS_use_Raw:nN #1 #2 {
22   #1 #2
23 }
24 \cs_new:Npn \BNVS_use_Raw:Nc #1 #2 {
25   \exp_args:Nnc \BNVS_use_Raw:NN #1 { #2 }
26 }
27 \cs_new:Npn \BNVS_use_Raw:nc #1 #2 {
28   \exp_last_unbraced:Nno
29   \BNVS_use_Raw:nN { #1 } { \cs:w #2 \cs_end: }
30 }
31 \cs_new:Npn \BNVS_use:Nc #1 #2 {
32   \BNVS_use_Raw:Nc #1 { \BNVS:c { #2 } }
33 }
34 \cs_new:Npn \BNVS_use:nc #1 #2 {
35   \BNVS_use_Raw:nc { #1 } { \BNVS:c { #2 } }
36 }
37 \cs_new:Npn \BNVS_tl_use_unbraced:nv #1 #2 {
38   \exp_last_unbraced:Nnf \use:n { #1 } { \BNVS_tl_use:nv {\use:n} { #2 } }
39 }
40 \cs_new:Npn \BNVS_tl_use:nvv #1 #2 {
41   \BNVS_tl_use:nv { \BNVS_tl_use:nv { #1 } { #2 } }
42 }
43 \cs_new:Npn \BNVS_tl_use:nvvv #1 #2 {
44   \BNVS_tl_use:nvv { \BNVS_tl_use:nv { #1 } { #2 } }
45 }
46 \cs_new:Npn \BNVS_tl_use:nvvvv #1 #2 {
47   \BNVS_tl_use:nvvv { \BNVS_tl_use:nv { #1 } { #2 } }
48 }
49 \cs_new:Npn \BNVS_log:n #1 { }
50 \cs_generate_variant:Nn \BNVS_log:n { x }

```

1.3.1 Debug

\BNVS_DEBUG_ON:	\BNVS_DEBUG_ON:
\BNVS_DEBUG_OFF:	\BNVS_DEBUG_OFF:
\BNVS_DEBUG_push:nn	\BNVS_DEBUG_push:nn {<types on>} {<types off>}
\BNVS_DEBUG_pop:	\BNVS_DEBUG_pop:

These functions activate or deactivate the debug mode. They are only active in the `beanoves-debug` package. Manage debug messaging for one given `<type>` or `<types>`. The implementation is not publicly exposed. The `<type>` is a single letter or `**`.

1.3.2 Facility layer: Functions

`\BNVS_new:cpn` `\BNVS_new:cpn` is like `\cs_new:cpn` except that the name argument is tagged for beanoves package. Similarly for `\BNVS_set:cpn`.

```
51 \cs_new:Npn \BNVS_new:cpn #1 {
52   \cs_new:cpn { \BNVS:c { #1 } }
53 }
54 \cs_new:Npn \BNVS_set:cpn #1 {
55   \cs_set:cpn { \BNVS:c { #1 } }
56 }
57 \cs_generate_variant:Nn \cs_generate_variant:Nn { c }
58 \cs_new:Npn \BNVS_generate_variant:cn #1 {
59   \cs_generate_variant:cn { \BNVS:c { #1 } }
60 }
```

1.3.3 logging

<code>\BNVS_warning:n</code>	<code>\BNVS_warning:n {<message>}</code>
<code>\BNVS_warning:x</code>	<code>\BNVS_error:n {<message>}</code>
<code>\BNVS_error:n</code>	<code>\BNVS_fatal:n {<message>}</code>
<code>\BNVS_error:x</code>	
<code>\BNVS_fatal:n</code>	Very rudimentary. No long message available for error recovery.
<code>\BNVS_fatal:x</code>	

```
61 \msg_new:nnn { beanoves } { :n } { #1 }
62 \msg_new:nnn { beanoves } { :nn } { #1~(#2) }
63 \cs_new:Npn \BNVS_warning:n {
64   \msg_warning:nnn { beanoves } { :n }
65 }
66 \cs_new:Npn \BNVS_warning:x {
67   \msg_warning:nnx { beanoves } { :n }
68 }
69 \cs_new:Npn \BNVS_error:n {
70   \msg_error:nnn { beanoves } { :n }
71 }
72 \cs_generate_variant:Nn \BNVS_error:n { x }
73 \cs_new:Npn \BNVS_fatal:n {
74   \msg_fatal:nnn { beanoves } { :n }
75 }
76 \cs_new:Npn \BNVS_fatal:x {
77   \msg_fatal:nnx { beanoves } { :n }
78 }
79 \cs_new:Npn \BNVS_fatal_unreachable: {
80   \BNVS_fatal:n { Unreachable }
81 }
82 \cs_new:Npn \BNVS_fatal_unreachable: { }
```

1.3.4 Facility layer: Variables

```
\BNVS_N_new:c \BNVS_N_new:c {<type>}
\BNVS_v_new:c \BNVS_v_new:c {<type>}
```

Creates a collection of typed utility functions, see usage below. These functions are undefined when no longer used. *<type>* is one of *tl*, *seq...*

```
83 \cs_new:Npn \BNVS_N_new:c #1 {
84   \cs_new:cpn { BNVS_#1:c } ##1 {
85     1 \BNVS:c{ ##1 } \tl_if_empty:nF { ##1 } { _ } #1
86   }
87   \cs_new:cpn { BNVS_#1_new:c } ##1 {
88     \use:c { #1_new:c } { \use:c { BNVS_#1:c } { ##1 } }
89   }
90   \cs_new:cpn { BNVS_#1_use:c } ##1 {
91     \use:c { \cs:w BNVS_#1:c \cs_end: { ##1 } }
92   }
93   \cs_new:cpn { BNVS_#1_use:Nc } ##1 ##2 {
94     \BNVS_use_Raw:Nc
95       ##1 { \cs:w BNVS_#1:c \cs_end: { ##2 } }
96   }
97   \cs_new:cpn { BNVS_#1_use:nc } ##1 ##2 {
98     \BNVS_use_Raw:nc
99       { ##1 } { \cs:w BNVS_#1:c \cs_end: { ##2 } }
100   }
101 }

102 \cs_new:Npn \BNVS_v_new:c #1 {
103   \cs_new:cpn { BNVS_#1_use:Nv } ##1 ##2 {
104     \exp_args:Nv ##1
105       { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
106   }
107   \cs_new:cpn { BNVS_#1_use:cv } ##1 ##2 {
108     \exp_args:Nv \BNVS_use:c { ##1 }
109       { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
110   }
111   \cs_new:cpn { BNVS_#1_use:nv } ##1 ##2 {
112     \exp_args:Nv \use:n { ##1 }
113       { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
114   }
115 }

116 \BNVS_N_new:c { bool }
117 \BNVS_N_new:c { int }
118 \BNVS_v_new:c { int }
119 \BNVS_N_new:c { tl }
120 \BNVS_v_new:c { tl }
121 \cs_new:Npn \BNVS_tl_use:Nvv #1 #2 {
122   \BNVS_tl_use:nv { \BNVS_tl_use:Nv #1 { #2 } }
123 }
```

```

124 \cs_new:Npn \BNVS_tl_use:Nvvv #1 #2 {
125   \BNVS_tl_use:nvv { \BNVS_tl_use:Nv #1 { #2 } }
126 }

127 \cs_new:Npn \BNVS_tl_use:Nvvvv #1 #2 {
128   \BNVS_tl_use:nvvv { \BNVS_tl_use:Nv #1 { #2 } }
129 }

130 \BNVS_N_new:c { str }
131 \BNVS_v_new:c { str }
132 \BNVS_N_new:c { seq }
133 \BNVS_v_new:c { seq }
134 \cs_undefine:N \BNVS_N_new:c

```

\BNVS_use:Ncn **\BNVS_use:Ncn** *<function>* {<core name>} {<type>}

```

135 \cs_new:Npn \BNVS_use:Ncn #1 #2 #3 {
136   \BNVS_use_Raw:c { BNVS_#3_use:Nc } #1 { #2 }
137 }

138 \cs_new:Npn \BNVS_use:ncn #1 #2 #3 {
139   \BNVS_use_Raw:c { BNVS_#3_use:nc } { #1 } { #2 }
140 }

141 \cs_new:Npn \BNVS_use:Nvn #1 #2 #3 {
142   \BNVS_use_Raw:c { BNVS_#3_use:Nv } #1 { #2 }
143 }

144 \cs_new:Npn \BNVS_use:nvn #1 #2 #3 {
145   \BNVS_use_Raw:c { BNVS_#3_use:nv } { #1 } { #2 }
146 }

147 \cs_new:Npn \BNVS_use:Ncnncn #1 #2 #3 {
148   \BNVS_use:ncn {
149     \BNVS_use:Ncn #1 { #2 } { #3 }
150   }
151 }

152 \cs_new:Npn \BNVS_use:ncnncn #1 #2 #3 {
153   \BNVS_use:ncn {
154     \BNVS_use:ncn { #1 } { #2 } { #3 }
155   }
156 }

157 \cs_new:Npn \BNVS_use:Nvnncn #1 #2 #3 {
158   \BNVS_use:ncn {
159     \BNVS_use:Nvn #1 { #2 } { #3 }
160   }
161 }

162 \cs_new:Npn \BNVS_use:nvnncn #1 #2 #3 {
163   \BNVS_use:ncn {
164     \BNVS_use:nvn { #1 } { #2 } { #3 }
165   }
166 }

```

```

167 \cs_new:Npn \BNVS_use:Ncncncn #1 #2 #3 #4 #5 {
168   \BNVS_use:ncn {
169     \BNVS_use:Ncncn   #1   { #2 } { #3 } { #4 } { #5 }
170   }
171 }

172 \cs_new:Npn \BNVS_use:ncncncn #1 #2 #3 #4 #5 {
173   \BNVS_use:ncn {
174     \BNVS_use:ncncn { #1 } { #2 } { #3 } { #4 } { #5 }
175   }
176 }

```

\BNVS_new_c:cn \BNVS_new_c:nc {<type>} {<core name>}

```

177 \cs_new:Npn \BNVS_new_c:nc #1 #2 {
178   \BNVS_new:cpn { #1_#2:c } {
179     \BNVS_use_Raw:c { BNVS_#1_use:nc } { \BNVS_use_Raw:c { #1_#2:N } }
180   }
181 }

182 \cs_new:Npn \BNVS_new_cn:nc #1 #2 {
183   \BNVS_new:cpn { #1_#2:cn } ##1 {
184     \BNVS_use:ncn { \BNVS_use_Raw:c { #1_#2:Nn } } { ##1 } { #1 }
185   }
186 }

187 \cs_new:Npn \BNVS_new_cnn:ncN #1 #2 #3 {
188   \BNVS_new:cpn { #2:cnn } ##1 {
189     \BNVS_use:Ncn { #3 } { ##1 } { #1 }
190   }
191 }

192 \cs_new:Npn \BNVS_new_cnn:nc #1 #2 {
193   \BNVS_use_Raw:nc {
194     \BNVS_new_cnn:ncN { #1 } { #1_#2 }
195   } { #1_#2:Nnn }
196 }

197 \cs_new:Npn \BNVS_new_cnv:ncN #1 #2 #3 {
198   \BNVS_new:cpn { #2:cnv } ##1 ##2 {
199     \BNVS_tl_use:nv {
200       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
201     }
202   }
203 }

204 \cs_new:Npn \BNVS_new_cnv:nc #1 #2 {
205   \BNVS_use_Raw:nc {
206     \BNVS_new_cnv:ncN { #1 } { #1_#2 }
207   } { #1_#2:Nnn }
208 }

```

```

209 \cs_new:Npn \BNVS_new_cnx:ncN #1 #2 #3 {
210   \BNVS_new_cpn { #2:cnx } ##1 ##2 {
211     \exp_args:Nnx \use:n {
212       \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 }
213     }
214   }
215 }

216 \cs_new:Npn \BNVS_new_cnx:nc #1 #2 {
217   \BNVS_use_Raw:nc {
218     \BNVS_new_cnx:ncN { #1 } { #1_#2 }
219   } { #1_#2:Nnn }
220 }

221 \cs_new:Npn \BNVS_new_cc:ncNn #1 #2 #3 #4 {
222   \BNVS_new_cpn { #2:cc } ##1 ##2 {
223     \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } { #4 }
224   }
225 }

226 \cs_new:Npn \BNVS_new_cc:ncn #1 #2 {
227   \BNVS_use_Raw:nc {
228     \BNVS_new_cc:ncNn { #1 } { #1_#2 }
229   } { #1_#2:NN }
230 }

231 \cs_new:Npn \BNVS_new_cc:nc #1 #2 {
232   \BNVS_new_cc:ncn { #1 } { #2 } { #1 }
233 }

234 \cs_new:Npn \BNVS_new_cn:ncNn #1 #2 #3 #4 {
235   \BNVS_new_cpn { #2:cn } ##1 {
236     \BNVS_use:Ncn #3 { ##1 } { #1 }
237   }
238 }

239 \cs_new:Npn \BNVS_new_cn:ncn #1 #2 {
240   \BNVS_use_Raw:nc {
241     \BNVS_new_cn:ncNn { #1 } { #1_#2 }
242   } { #1_#2:Nn }
243 }

244 \cs_new:Npn \BNVS_new_cv:ncNn #1 #2 #3 #4 {
245   \BNVS_new_cpn { #2:cv } ##1 ##2 {
246     \BNVS_use:nvn {
247       \BNVS_use:Ncn #3 { ##1 } { #1 }
248     } { ##2 } { #4 }
249   }
250 }

251 \cs_new:Npn \BNVS_new_cv:ncn #1 #2 {
252   \BNVS_use_Raw:nc {
253     \BNVS_new_cv:ncNn { #1 } { #1_#2 }
254   } { #1_#2:Nn }
255 }

```



```

256 \cs_new:Npn \BNVS_new_cv:nc #1 #2 {
257   \BNVS_new_cv:ncn { #1 } { #2 } { #1 }
258 }

259 \cs_new:Npn \BNVS_l_use:Ncn #1 #2 #3 {
260   \BNVS_use_Raw:Nc #1 { \BNVS_l:cn { #2 } { #3 } }
261 }

262 \cs_new:Npn \BNVS_l_use:ncn #1 #2 #3 {
263   \BNVS_use_Raw:nc { #1 } { \BNVS_l:cn { #2 } { #3 } }
264 }

265 \cs_new:Npn \BNVS_g_use:Ncn #1 #2 #3 {
266   \BNVS_use_Raw:Nc #1 { \BNVS_g:cn { #2 } { #3 } }
267 }

268 \cs_new:Npn \BNVS_g_use:ncn #1 #2 #3 {
269   \BNVS_use_Raw:nc { #1 } { \BNVS_g:cn { #2 } { #3 } }
270 }

```

```

\BNVS_new_conditional:cpnn \BNVS_new_conditional:cpnn {<core>} <parameter> {<conditions>} {<code>}

```

```

271 \cs_generate_variant:Nn \prg_new_conditional:Npnn { c }

272 \cs_new:Npn \BNVS_new_conditional:cpnn #1 {
273   \prg_new_conditional:cpnn { \BNVS:c { #1 } }
274 }

275 \cs_generate_variant:Nn \prg_generate_conditional_variant:Nnn { c }
276 \cs_new:Npn \BNVS_generate_conditional_variant:cnn #1 {
277   \prg_generate_conditional_variant:cnn { \BNVS:c { #1 } }
278 }

279 \cs_new:Npn \BNVS_new_conditional_vn:cNnn #1 #2 #3 #4 {
280   \BNVS_new_conditional:cpnn { #1:vn } ##1 ##2 { #4 } {
281     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
282       \prg_return_true:
283     } {
284       \prg_return_false:
285     }
286   }
287 }

288 \cs_new:Npn \BNVS_new_conditional_vn:cnn #1 #2 {
289   \BNVS_use:nc {
290     \BNVS_new_conditional_vn:cNnn { #1 }
291   } { #1:nn TF } { #2 }
292 }

293 \cs_new:Npn \BNVS_new_conditional_vc:cNnn #1 #2 #3 #4 {
294   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #4 } {
295     \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
296       \prg_return_true:
297     } {
298       \prg_return_false:
299     }
300   }
301 }

```

```

302 \cs_new:Npn \BNVS_new_conditional_vc:cnn #1 {
303   \BNVS_use:nc {
304     \BNVS_new_conditional_vc:cNnn { #1 }
305   } { #1:ncTF }
306 }

307 \cs_new:Npn \BNVS_new_conditional_vvc:cNnnn #1 #2 #3 #4 #5 {
308   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #5 } {
309     \BNVS_use:nvn {
310       \BNVS_use:Nvn #2 { ##1 } { #3 }
311     } { ##2 } { #4 } { ##3 } {
312       \prg_return_true:
313     } {
314       \prg_return_false:
315     }
316   }
317 }

318 \cs_new:Npn \BNVS_new_conditional_vvc:cnnn #1 {
319   \BNVS_use:nc {
320     \BNVS_new_conditional_vvc:cNnnn { #1 }
321   } { #1:nncTF }
322 }

323 \cs_new:Npn \BNVS_new_conditional_vc:cNn #1 #2 #3 {
324   \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #3 } {
325     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } {
326       \prg_return_true:
327     } {
328       \prg_return_false:
329     }
330   }
331 }

332 \cs_new:Npn \BNVS_new_conditional_vc:cn #1 {
333   \BNVS_use:nc {
334     \BNVS_new_conditional_vc:cNn { #1 }
335   } { #1:ncTF }
336 }

337 \cs_new:Npn \BNVS_new_conditional_vvc:cNn #1 #2 #3 {
338   \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #3 } {
339     \BNVS_tl_use:nv {
340       \BNVS_tl_use:Nv #2 { ##1 }
341     } { ##2 } { ##3 } {
342       \prg_return_true:
343     } {
344       \prg_return_false:
345     }
346   }
347 }

```

```

348 \cs_new:Npn \BNVS_new_conditional_vvc:cn #1 {
349   \BNVS_use:nc {
350     \BNVS_new_conditional_vvc:cNn { #1 }
351   } { #1:nncTF }
352 }

```

1.3.5 Regex

```

353 \cs_new:Npn \BNVS_regex_use:Nc #1 #2 {
354   \BNVS_use_Raw:Nc #1 { c \BNVS:c { #2 } _regex }
355 }

```

1.3.6 Token lists

<u>__bnvs_tl_clear:c</u>	__bnvs_tl_clear:c {<core key tl>}
__bnvs_tl_clear_new:c	__bnvs_tl_use:c {<core>}
__bnvs_tl_new	__bnvs_tl_count:c {<core>}
__bnvs_tl_set_eq:cc	__bnvs_tl_set_eq:cc {<lhs core name>} {<rhs core name>}
__bnvs_tl_set:cn	__bnvs_tl_set:cn {<core>} {<tl>}
__bnvs_tl_set:(cv cx ce co)	__bnvs_tl_set:cv {<core>} {<value core name>}
__bnvs_tl_put_left:cn	__bnvs_tl_put_left:cn {<core>} {<tl>}
__bnvs_tl_put_right:cn	__bnvs_tl_put_right:cn {<core>} {<tl>}
__bnvs_tl_put_right:(cx ce cv co)	__bnvs_tl_put_right:cv {<core>} {<value core name>}
	__bnvs_tl_put_right:co {<core>} {<tl>}

These are shortcuts to

- \tl_clear:c {l__bnvs_<core>_tl}
- \tl_clear_new:c {l__bnvs_<core>_tl}
- \tl_use:c {l__bnvs_<core>_tl}
- \tl_set_eq:cc {l__bnvs_<lhs core>_tl} {l__bnvs_<rhs core>_tl}
- \tl_set:cv {l__bnvs_<core>_tl} {l__bnvs_<value core>_tl}
- \tl_set:cx {l__bnvs_<core>_tl} {<tl>}
- \tl_set:co {l__bnvs_<core>_tl} {<tl>}
- \tl_put_left:cn {l__bnvs_<core>_tl} {<tl>}
- \tl_put_right:cn {l__bnvs_<core>_tl} {<tl>}
- \tl_put_right:cv {l__bnvs_<core>_tl} {l__bnvs_<value core>_tl}
- \tl_put_right:co {l__bnvs_<core>_tl} {<tl>}

__bnvs_tl_clear_all:n __bnvs_tl_clear_all:n {<cores>}

Run __bnvs_tl_clear:c for each individual item of <cores>.

```

356 \BNVS_new:cpn { tl_clear_all:n } #1 {
357   \tl_map_inline:nn { #1 } {
358     \__bnvs_tl_clear:c { ##1 }
359   }
360 }

```

`\BNVS_new_conditional_vnc:cn` `\BNVS_new_conditional_vnc:cn {<core>} {<conditions>}`

Creates `<core>:vncTF` from `<core>:nncTF`.

```

361 \cs_new:Npn \BNVS_new_conditional_vnc:cNn #1 #2 #3 {
362   \BNVS_new_conditional:cpnn { #1:vnc } ##1 ##2 ##3 { #3 } {
363     \BNVS_tl_use:Nv #2 { ##1 } { ##2 } { ##3 } {
364       \prg_return_true:
365     } {
366       \prg_return_false:
367     }
368   }
369 }

370 \cs_new:Npn \BNVS_new_conditional_vnc:cn #1 {
371   \BNVS_use:nc {
372     \BNVS_new_conditional_vnc:cNn { #1 }
373   } { #1:nncTF }
374 }

```

`\BNVS_new_conditional_vvnc:cn` `\BNVS_new_conditional_vvnc:cn {<core>} {<conditions>}`

Creates `<core>:vvncTF` from `<core>:nnncTF`.

```

375 \cs_new:Npn \BNVS_new_conditional_vvnc:cNn #1 #2 #3 {
376   \BNVS_new_conditional:cpnn { #1:vvnc } ##1 ##2 ##3 ##4 { #3 } {
377     \BNVS_tl_use:nv {
378       \BNVS_tl_use:Nv #2 { ##1 }
379     } { ##2 } { ##3 } { ##4 } {
380       \prg_return_true:
381     } {
382       \prg_return_false:
383     }
384   }
385 }

386 \cs_new:Npn \BNVS_new_conditional_vvnc:cn #1 {
387   \BNVS_use:nc {
388     \BNVS_new_conditional_vvnc:cNn { #1 }
389   } { #1:nnncTF }
390 }

```

```

391 \cs_new:Npn \BNVS_new_conditional_vvvc:cNn #1 #2 #3 {
392   \BNVS_new_conditional:cpnn { #1:vvvc } ##1 ##2 ##3 ##4 { #3 } {
393     \BNVS_tl_use:nvv {
394       \BNVS_tl_use:Nv #2 { ##1 }
395     } { ##2 } { ##3 } { ##4 } {
396       \prg_return_true:
397     } {
398       \prg_return_false:
399     }
400   }
401 }

```

`\BNVS_new_conditional_vvvc:cn` `\BNVS_new_conditional_vvvc:cn {<core>} {<conditions>}`

Creates `<core>:vvvcTF` from `<core>:nnncTF`.

```

402 \cs_new:Npn \BNVS_new_conditional_vvvc:cn #1 {
403   \BNVS_use:nc {
404     \BNVS_new_conditional_vvvc:cNn { #1 }
405   } { #1:nnncTF }
406 }

407 \cs_new:Npn \BNVS_new_tl_c:c {
408   \BNVS_new_c:nc { tl }
409 }
410 \BNVS_new_tl_c:c { clear }
411 \BNVS_new_tl_c:c { clear_new }
412 \BNVS_new_tl_c:c { use }
413 \BNVS_new_tl_c:c { count }

414 \BNVS_new:cpn { tl_set:eq:cc } #1 #2 {
415   \BNVS_use:ncncn { \tl_set:eq:NN } { #1 } { tl } { #2 } { tl }
416 }

417 \cs_new:Npn \BNVS_new_tl_cn:c {
418   \BNVS_new_cn:nc { tl }
419 }

420 \cs_new:Npn \BNVS_new_tl_cv:c #1 {
421   \BNVS_new_cv:ncn { tl } { #1 } { tl }
422 }
423 \BNVS_new_tl_cn:c { set }
424 \BNVS_new_tl_cv:c { set }

425 \BNVS_new:cpn { tl_set:cx } {
426   \exp_args:Nnx \__bnvs_tl_set:cn
427 }

428 \BNVS_new:cpn { tl_set:ce } {
429   \exp_args:Nne \__bnvs_tl_set:cn
430 }

431 \BNVS_new:cpn { tl_set:co } {
432   \exp_args:Nno \__bnvs_tl_set:cn
433 }

```

```

434 \BNVS_new_tl_cn:c { put_right }
435 \BNVS_new_tl_cv:c { put_right }
436 % \BNVS_generate_variant:cn { tl_put_right:cn } { cx }

437 \BNVS_new:cpn { tl_put_right:cx } {
438   \exp_args:Nnnx \BNVS_use:c { tl_put_right:cn }
439 }

440 \BNVS_new:cpn { tl_put_right:ce } {
441   \exp_args:Nnne \BNVS_use:c { tl_put_right:cn }
442 }
443 \BNVS_new:cpn { tl_put_right:co } {
444   \exp_args:Nnno \BNVS_use:c { tl_put_right:cn }
445 }
446 \BNVS_new_tl_cn:c { put_left }
447 \BNVS_new_tl_cv:c { put_left }
448 % \BNVS_generate_variant:cn { tl_put_left:cn } { cx }
449 % \begin{KWN.test}{bnvs:c={ tl_put_right:co }, eringo}
450 % \__bnvs_tl_set:cn { TEST_A } { JES }
451 % \__bnvs_tl_set:co { TEST_B } { { \l__bnvs_TEST_A_tl } }
452 % \assert_equal_tl:vn { TEST_B } { { JES } } A
453 % \end{KWN.test}

454 \BNVS_new:cpn { tl_put_left:cx } {
455   \exp_args:Nnnx \BNVS_use:c { tl_put_left:cn }
456 }

```

```

\__bnvs_tl_if_exist:cTF \__bnvs_tl_if_empty:cTF {<core>} {<yes>} {<no>}
\__bnvs_tl_if_empty:cTF \__bnvs_tl_if_exist:cTF {<core>} {<yes>} {<no>}
\__bnvs_tl_if_blank:vTF \__bnvs_tl_if_blank:vTF {<core>} {<yes>} {<no>}
\__bnvs_tl_if_eq:cnTF \__bnvs_tl_if_eq:cnTF {<core>} {<tl>} {<yes>} {<no>}

```

These are shortcuts to

- \tl_if_exist:cTF {l__bnvs_<core>_tl} {<yes>} {<no>}
- \tl_if_empty:cTF {l__bnvs_<core>_tl} {<yes>} {<no>}
- \tl_if_eq:cnTF {l__bnvs_<core>_tl}{<tl>} {<yes>} {<no>}

```

457 \cs_new:Npn \BNVS_new_conditional_c:ncNn #1 #2 #3 #4 {
458   \BNVS_new_conditional:cpnn { #2 } ##1 { #4 } {
459     \BNVS_use:Ncn #3 { ##1 } { #1 } {
460       \prg_return_true:
461     } {
462       \prg_return_false:
463     }
464   }
465 }

```

```

466 \cs_new:Npn \BNVS_new_conditional_c:ncn #1 #2 {
467   \BNVS_use_Raw:nc {
468     \BNVS_new_conditional_c:ncNn { #1 } { #1_#2:c }
469   } { #1_#2:Ntf }
470 }
471 \BNVS_new_conditional_c:ncn { t1 } { if_empty } { p, T, F, TF }
472 \BNVS_new_conditional_c:ncn { t1 } { if_exist } { p, T, F, TF }

473 \BNVS_new_conditional:cpnn { t1_if_blank:v } #1 { T, F, TF } {
474   \BNVS_tl_use:Nv \t1_if_blank:nTF { #1 } {
475     \prg_return_true:
476   } {
477     \prg_return_false:
478   }
479 }

480 \cs_new:Npn \BNVS_new_conditional_cn:ncNn #1 #2 #3 #4 {
481   \BNVS_new_conditional:cpnn { #2:cn } ##1 ##2 { #4 } {
482     \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } {
483       \prg_return_true:
484     } {
485       \prg_return_false:
486     }
487   }
488 }

489 \cs_new:Npn \BNVS_new_conditional_cn:ncn #1 #2 {
490   \BNVS_use_Raw:nc {
491     \BNVS_new_conditional_cn:ncNn { #1 } { #1_#2 }
492   } { #1_#2:NnTF }
493 }
494 \BNVS_new_conditional_cn:ncn { t1 } { if_eq } { T, F, TF }

495 \cs_new:Npn \BNVS_new_conditional_cv:ncNn #1 #2 #3 #4 {
496   \BNVS_new_conditional:cpnn { #2:cv } ##1 ##2 { #4 } {
497     \BNVS_use:nvn {
498       \BNVS_use:Ncn #3 { ##1 } { #1 }
499       } { ##2 } { #1 } {
500       \prg_return_true:
501     } {
502       \prg_return_false:
503     }
504   }
505 }

506 \cs_new:Npn \BNVS_new_conditional_cv:ncn #1 #2 {
507   \BNVS_use_Raw:nc {
508     \BNVS_new_conditional_cv:ncNn { #1 } { #1_#2 }
509   } { #1_#2:NnTF }
510 }
511 \BNVS_new_conditional_cv:ncn { t1 } { if_eq } { T, F, TF }

512 \BNVS_new:cpn { gput_right:cccn } #1 #2 #3 {
513   \tl_gput_right:cn { \_bnvs_g_IPK:w { #1 } { #2 } { #3 } }
514 }

```

1.3.7 Strings

`\_bnvs_str_if_eq:vvTF` `\_bnvs_str_if_eq:vvTF {<core>} {<tl>} {<yes:>} {<no:>}`

These are shortcuts to

- `\str_if_eq:ccTF` `l\_bnvs_<core>_tl` `l\_bnvs_<core>_tl {<yes:>} {<no:>}`

```

515 \cs_new:Npn \BNVS_new_conditional_vv:cnN #1 #2 #3 {
516   \BNVS_new_conditional:cpnn { #1:vv } ##1 ##2 { #3 } {
517     \BNVS_tl_use:nv {
518       \BNVS_tl_use:Nv #2 { ##1 }
519     } { ##2 } {
520       \prg_return_true:
521     } {
522       \prg_return_false:
523     }
524   }
525 }

526 \cs_new:Npn \BNVS_new_conditional_vv:cn #1 {
527   \BNVS_use:nc {
528     \BNVS_new_conditional_vvnc:cnN { #1 }
529   } { #1:nnTF }
530 }

531 \cs_new:Npn \BNVS_new_conditional_vn:cnN #1 #2 #3 #4 {
532   \BNVS_new_conditional:cpnn { #2:vn } ##1 ##2 { #4 } {
533     \BNVS_use:Nvn #3 { ##1 } { #1 } { ##2 } {
534       \prg_return_true:
535     } {
536       \prg_return_false:
537     }
538   }
539 }

540 \cs_new:Npn \BNVS_new_conditional_vn:ncn #1 #2 {
541   \BNVS_use_Raw:nc {
542     \BNVS_new_conditional_vn:cnN { #1 } { #1_#2 }
543   } { #1_#2:nnTF }
544 }
545 \BNVS_new_conditional_vn:ncn { str } { if_eq } { T, F, TF }

546 \cs_new:Npn \BNVS_new_conditional_vv:cnN #1 #2 #3 #4 {
547   \BNVS_new_conditional:cpnn { #2:vv } ##1 ##2 { #4 } {
548     \BNVS_use:nvn {
549       \BNVS_use:Nvn #3 { ##1 } { #1 }
550     } { ##2 } { #1 } {
551       \prg_return_true:
552     } {
553       \prg_return_false:
554     }
555   }
556 }
```



```

557 \cs_new:Npn \BNVS_new_conditional_vv:ncn #1 #2 {
558   \BNVS_use_Raw:nc {
559     \BNVS_new_conditional_vv:ncNn { #1 } { #1_#2 }
560   } { #1_#2:nnTF }
561 }
562 \BNVS_new_conditional_vv:ncn { str } { if_eq } { T, F, TF }

```

1.3.8 Sequences

__bnvs_seq_count:c	__bnvs_seq_new:c {<core>}
__bnvs_seq_clear:c	__bnvs_seq_count:c {<core>}
__bnvs_seq_set_eq:cc	__bnvs_seq_clear:c {<core>}
__bnvs_seq_gset_eq:cc	__bnvs_seq_set_eq:cc {<core ₁ >} {<core ₂ >}
__bnvs_seq_use:cn	__bnvs_seq_use:cn {<core>} {<separator>}
__bnvs_seq_item:cn	__bnvs_seq_item:cn {<core>} {<integer expression>}
__bnvs_seq_remove_all:cn	__bnvs_seq_remove_all:cn {<core>} {<t1>}
__bnvs_seq_put_left:cn	__bnvs_seq_put_right:cn {<seq core>} {<t1>}
__bnvs_seq_put_left:cv	__bnvs_seq_put_right:cv {<seq core>} {<t1 core>}
__bnvs_seq_put_right:cn	__bnvs_seq_set_split:cn {<seq core>} {<t1>} {<separator>}
__bnvs_seq_put_right:cv	__bnvs_seq_pop_left:cc {<core ₁ >} {<core ₂ >}
__bnvs_seq_set_split:cn	
__bnvs_seq_set_split:(cnv cnx)	
__bnvs_seq_pop_left:cc	

These are shortcuts to

- \seq_set_eq:cc {l__bnvs_<core₁>_seq} {l__bnvs_<core₂>_seq}
- \seq_count:c {l__bnvs_<core>_seq}
- \seq_use:cn {l__bnvs_<core>_seq} {<separator>}
- \seq_item:cn {l__bnvs_<core>_seq} {<integer expression>}
- \seq_remove_all:cn {l__bnvs_<core>_seq} {<t1>}
- \seq_clear:c {l__bnvs_<core>_seq}
- \seq_put_right:cv {l__bnvs_<core>_seq} {l__bnvs_<core>_t1}
- \seq_set_split:cn {l__bnvs_<core>_seq} {l__bnvs_<core>_t1}\KWNmeta{t1}

```

563 \BNVS_new_c:nc { seq } { count }
564 \BNVS_new_c:nc { seq } { clear }
565 \BNVS_new_cn:nc { seq } { use }
566 \BNVS_new_cn:nc { seq } { item }
567 \BNVS_new_cn:nc { seq } { remove_all }
568 \BNVS_new_cn:nc { seq } { map_inline }
569 \BNVS_new_cc:nc { seq } { set_eq }
570 \BNVS_new_cc:nc { seq } { gset_eq }
571 \BNVS_new_cn:ncn { seq } { put_left } { t1 }
572 \BNVS_new_cv:ncn { seq } { put_left } { t1 }

```

```

573 \BNVS_new_cn:ncn { seq } { put_right } { t1 }
574 \BNVS_new_cv:ncn { seq } { put_right } { t1 }
575 \BNVS_new_cnn:nc { seq } { set_split }
576 \BNVS_new_cnv:nc { seq } { set_split }
577 \BNVS_new_cnx:nc { seq } { set_split }
578 \BNVS_new_cc:ncn { seq } { pop_left } { t1 }
579 \BNVS_new_cc:ncn { seq } { pop_right } { t1 }

```

```

__bnvs_seq_if_empty:cTF  __bnvs_seq_if_empty:cTF {<seq core>} {<yes:>} {<no:>}
__bnvs_seq_get_right:ccTF __bnvs_seq_get_right:ccTF {<seq core>} {<t1 core>} {<yes:>} {<no:>}
__bnvs_seq_pop_left:ccTF
__bnvs_seq_pop_right:ccTF

```

```

580 \cs_new:Npn \BNVS_new_conditional_cc:ncnn #1 #2 #3 #4 {
581   \BNVS_new_conditional:cpnn { #1_#2:cc } ##1 ##2 { #4 } {
582     \BNVS_use:ncncn {
583       \BNVS_use_Raw:c { #1_#2:NNTF }
584     } { ##1 } { #1 } { ##2 } { #3 } {
585       \prg_return_true:
586     } {
587       \prg_return_false:
588     }
589   }
590 }
591 \BNVS_new_conditional_c:ncn { seq } { if_empty } { T, F, TF }
592 \BNVS_new_conditional_cc:ncnn
593   { seq } { get_right } { t1 } { T, F, TF }
594 \BNVS_new_conditional_cc:ncnn
595   { seq } { pop_left } { t1 } { T, F, TF }
596 \BNVS_new_conditional_cc:ncnn
597   { seq } { pop_right } { t1 } { T, F, TF }

```

1.3.9 Integers

```

__bnvs_int_new:c  __bnvs_int_new:c {<core>}
__bnvs_int_use:c  __bnvs_int_use:c {<core>}
__bnvs_int_zero:c __bnvs_int_incr:c {<core>}
__bnvs_int_inc:c  __bnvs_int_decr:c {<core>}
__bnvs_int_decr:c __bnvs_int_set:cn {<core>} {<value>}
__bnvs_int_set:cn
__bnvs_int_set:cv

```

These are shortcuts to

- `\int_new:c {l__bnvs_<core>_int}`
- `\int_use:c {l__bnvs_<core>_int}`
- `\int_incr:c {l__bnvs_<core>_int}`
- `\int_decr:c {l__bnvs_<core>_int}`
- `\int_set:cn {l__bnvs_<core>_int} {<value>}`

```

598 \BNVS_new_c:nc { int } { new }
599 \BNVS_new_c:nc { int } { use }
600 \BNVS_new_c:nc { int } { zero }
601 \BNVS_new_c:nc { int } { incr }
602 \BNVS_new_c:nc { int } { decr }
603 \BNVS_new_cn:nc { int } { set }
604 \BNVS_new_cv:ncn { int } { set } { int }

```

1.4 Debug facilities

Typesetting file `beanoves.dtx` creates both `beanoves` and `beanoves-debug` style files. The former is intended for everyday use whereas the latter contains supplemental debugging and testing facilities which are intentionally left undocumented. For example, we have aliases for `\group_begin:` and `\group_end:` to allow the display of supplemental informations while debugging. We also have some techniques for coverage as well as inline testing.

1.5 Local variables

We make heavy use of local variables and function scopes. Many functions are executed within a `TEX` group, which ensures no name collision with the caller stack. The number of variables used has not been optimized, nor the `TEX` groups used. Optimization often goes against readability. Next local variables are used by different functions. These are one word letter variables.

`\l__bnvs_A_tl:` ,

`\l__bnvs_Z_tl:` ,

`\l__bnvs_L_tl:` Start, end and lentgh of a range

```

605 \tl_new:N \l__bnvs_A_tl
606 \tl_new:N \l__bnvs_Z_tl
607 \tl_new:N \l__bnvs_L_tl

```

`\l__bnvs_C_tl:` A function name in a call, as regex caught group

`\l__bnvs_I_tl:` A frame identifier

`\l__bnvs_J_tl:` The current frame identifier

`\l__bnvs_S_tl:` A short name, a regex caught group.

`\l__bnvs_R_tl:` A relative, a regex caught group.

`\l__bnvs_P_tl:` A path, a regex caught group.

`\l__bnvs_M_tl:` An assignment modifier, a regex caught group.

```

608 \tl_new:N \l__bnvs_I_tl
609 \tl_new:N \l__bnvs_J_tl
610 \tl_new:N \l__bnvs_K_tl

```

```

611 \tl_new:N \l__bnvs_S_tl
612 \tl_new:N \l__bnvs_R_tl
613 \tl_new:N \l__bnvs_P_tl
614 \tl_new:N \l__bnvs_M_tl

```

\l__bnvs_scanned_tl: To store the scanned material.

```

615 \tl_new:N \l__bnvs_xprt_tl
616 \tl_new:N \l__bnvs_xprted_Raw_tl
617 \tl_new:N \l__bnvs_xprted_tl

```

\l__bnvs_N_tl: The representation of an unsigned decimal integer.

```

618 \tl_new:N \l__bnvs_N_tl

```

\l__bnvs_V_tl: A number value as opposit of a range value.

```

619 \tl_new:N \l__bnvs_V_tl

```

\l__bnvs_H_tl: The left hand side of an assignment.

```

620 \tl_new:N \l__bnvs_H_tl

```

```

621 \tl_new:N \l__bnvs_B_tl
622 \tl_new:N \l__bnvs_T_tl
623 \tl_new:N \l__bnvs_U_tl
624 \tl_new:N \l__bnvs_F_tl
625 \tl_new:N \l__bnvs_Q_tl
626 \tl_new:N \l__bnvs_a_tl
627 \tl_new:N \l__bnvs_z_tl
628 \tl_new:N \l__bnvs_l_tl
629 \tl_new:N \l__bnvs_r_tl
630 \tl_new:N \l__bnvs_n_tl
631 \tl_new:N \l__bnvs_ref_tl
632 \tl_new:N \l__bnvs_v_tl
633 \tl_new:N \l__bnvs_b_tl
634 \tl_new:N \l__bnvs_c_tl
635 \tl_new:N \l__bnvs_ans_tl
636 \tl_new:N \l__bnvs_base_tl
637 \tl_new:N \l__bnvs_group_tl
638 \tl_new:N \l__bnvs_scan_tl
639 \tl_new:N \l__bnvs_query_tl
640 \tl_new:N \l__bnvs_n_incr_tl
641 \tl_new:N \l__bnvs_incr_tl
642 \tl_new:N \l__bnvs_plus_tl
643 \tl_new:N \l__bnvs_rhs_tl
644 \tl_new:N \l__bnvs_post_tl
645 \tl_new:N \l__bnvs_suffix_tl
646 \tl_new:N \l__bnvs_index_tl
647 \int_new:N \g__bnvs_call_int
648 \int_new:N \l__bnvs_i_int
649 \seq_new:N \g__bnvs_def_seq
650 \seq_new:N \l__bnvs_a_seq

```

```

651 \seq_new:N \l__bnvs_b_seq
652 \seq_new:N \l__bnvs_ans_seq
653 \seq_new:N \l__bnvs_Q_seq
654 \seq_new:N \l__bnvs_R_seq
655 \seq_new:N \l__bnvs_match_seq
656 \seq_new:N \l__bnvs_split_seq
657 \seq_new:N \l__bnvs_P_seq
658 \bool_new:N \l__bnvs_parse_bool
659 \bool_set_false:N \l__bnvs_parse_bool
660 \bool_new:N \l__bnvs_deep_bool
661 \bool_set_false:N \l__bnvs_deep_bool
662 \bool_new:N \l__bnvs_no_range_bool
663 \bool_set_false:N \l__bnvs_no_range_bool

664 \BNVS_new:cpn { tl_clear_ans: } {
665   \__bnvs_tl_clear:c { ans }
666 }

667 \cs_new:Npn \BNVS_error_ans:x {
668   \__bnvs_tl_put_right:cn { ans } 0
669   \BNVS_error:x
670 }

```

In order to implement the provide feature, we add getters and setters

```

671 \BNVS_new:cpn { set_true:c } #1 {
672   \exp_args:Nc \bool_set_true:N { \BNVS_l:cn { #1 } { bool } }
673 }

674 \BNVS_new:cpn { set_false:c } #1 {
675   \exp_args:Nc \bool_set_false:N { \BNVS_l:cn { #1 } { bool } }
676 }

```

1.6 Infinite loop management

Unending recursivity is managed here.

`\g__bnvs_call_int` Some functions calls, as well as some loop bodies, decrement this counter. When this counter reaches 0, an error is raised or a computation is aborted.

(End of definition for \g__bnvs_call_int.)

```

677 \int_const:Nn \c__bnvs_max_call_int { 8192 }

```

`\__bnvs_greset_call:` `\__bnvs_greset_call:`

Reset globally the call stack counter to its maximum value.

```

678 \BNVS_new:cpn { greset_call: } {
679   \int_gset:Nn \g__bnvs_call_int { \c__bnvs_max_call_int }
680 }

```

```

__bnvs_if_call:TF \__bnvs_call_do:TF {<yes:>} {<no:>}

```

Decrement the `\g__bnvs_call_int` counter globally and execute `<yes:>` if we have not reached 0, `<no:>` otherwise.

```

681 \BNVS_new_conditional:cpnn { if_call: } { T, F, TF } {
682   \int_gdecr:N \g__bnvs_call_int
683   \int_compare:nNnTF \g__bnvs_call_int > 0 {
684     \prg_return_true:
685   } {
686     \prg_return_false:
687   }
688 }

```

1.7 Overlay specification

1.7.1 Registration

We keep track of the `<id> ! <path>` combinations and provide looping mechanisms. Hereafter, `<id>`, `<path>` and `<key>` always represent pure strings.

```

\__bnvs_g_IPK:w \__bnvs_g_IPK:w {<id>} {<path>} {<key>}
\__bnvs_g_IP:w \__bnvs_g_IP:w {<id>} {<path>}
\__bnvs_g_I:w \__bnvs_g_I:w {<id>}

```

Create a unique global name from the arguments.

```

689 \BNVS_new:cpn { g_IPK:w } #1 #2 #3 { g__bnvs@#1!#2/#3_t1 }
690 \BNVS_new:cpn { g_IP:w } #1 #2 { g__bnvs@#1!#2_keys_seq }
691 \BNVS_new:cpn { g_I:w } #1 { g__bnvs@#1_paths_seq }

```

`\g__bnvs_I_seq` Global list of globally registered identifiers.

(End of definition for \g__bnvs_I_seq.)

```

692 \seq_new:N \g__bnvs_I_seq

```

1.7.2 Data model

The whole data model is somehow similar to a hierarchical file system: the `<id>` corresponds to the drive, the `<path>` points to a directory, each directory may contain individual files. File named “=” contains initial data whereas file named “@” contains the static resolved data. The latter is initially undefined and becomes empty as soon as the initial data is defined or set. In order to allow index override, files respectively named “`<N>`” and “`@<N>`” are used similarly, `<N>` denoting a normalized integer (no leading 0, no leading + sign). The dot (.) is used to separate components of `<path>`.

1.7.3 Low level IPK model functions

These are `gset` and `gunset` functions. Each `gset` function must be balanced by a `gunset` function to prevent “memory leaks”.

```

\__bnvs_gset:nnnn \__bnvs_gset:nnnn {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨def⟩}
\__bnvs_gset:(nnnv|nnne|vnnn|nvnn|nvvn|vvnn|vvvn)

```

Manage the storage. Keep track of all the `⟨path⟩`’s for a given `⟨id⟩` and all the `⟨key⟩`’s for a given `⟨id⟩ ! ⟨path⟩` combination.

Implementation details, next macros are defined lazily at the global level:

- `\__bnvs_gset@⟨id⟩ ! ⟨path⟩ :nn {⟨key⟩} {⟨def⟩}`,
- `\__bnvs_gset@⟨id⟩ :nnn {⟨path⟩} {⟨key⟩} {⟨def⟩}`,
- `\g__bnvs@⟨id⟩ !_paths_tl`, records all `⟨path⟩` for `⟨id⟩`,
- `\g__bnvs@⟨id⟩ ! ⟨path⟩ _keys_tl`, records all `⟨key⟩` for `⟨id⟩ ! ⟨path⟩`,
- `\g__bnvs@⟨id⟩ ! ⟨path⟩ _keys_tl`,
- `\g__bnvs@⟨id⟩ ! ⟨path⟩ / ⟨key⟩ _tl`, record the `⟨def⟩` for the `⟨id⟩ ! ⟨path⟩ / ⟨key⟩` combination.

```

693 \BNVS_new:cpn { gset:Nn } #1 {
694   \tl_gclear_new:N #1
695   \tl_gset:Nn #1
696 }
697 \BNVS_new:cpn { gset_IPKV:NNw } #1 #2 #3 #4 {
698   \seq_gclear_new:N #1
699   \cs_new:Npn #2 ##1 ##2 {
700     \seq_if_in:NnF #1 { ##1 } {
701       \seq_gput_right:Nn #1 { ##1 }
702     }
703     \exp_args:Nc \__bnvs_gset:Nn {
704       \__bnvs_g_IPK:w { #3 } { #4 } { ##1 }
705     } { ##2 }
706   }
707   #2
708 }
709 \BNVS_new:cpn { gset:NNnnnn } #1 #2 #3 #4 {
710   \cs_if_exist_use:NF #1 {
711     \seq_gput_right:Nn \g__bnvs_I_seq { #3 }
712     \seq_gclear_new:N #2
713     \cs_new:Npn #1 ##1 {
714       \seq_if_in:NnF #2 { ##1 } {
715         \seq_gput_right:Nn #2 { ##1 }
716       }
717       \exp_args:Ncc \__bnvs_gset_IPKV:NNw
718       { \__bnvs_g_IP:w { #3 } { ##1 } }
719       { __bnvs_gset@#3!##1:nn } { #3 } { ##1 }
720     }
721     #1
722   }

```

```

723   { #4 }
724 }
725 \BNVS_new:cpn { gset:nnnn } #1 #2 #3 #4 {
726   \cs_if_exist_use:cF { __bnvs_gset@#1!#2:nn } {
727     \exp_args:Ncc \__bnvs_gset:NNnnnn
728     { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
729   } { #3 } { #4 }
730 }
731 \BNVS_new:cpn { gset:vnnn } {
732   \BNVS_tl_use:cv { gset:nnnn }
733 }
734 \BNVS_new:cpn { gset:nvnn } #1 {
735   \BNVS_tl_use:nv { \__bnvs_gset:nnnn { #1 } }
736 }
737 \BNVS_new:cpn { gset:vvnn } {
738   \BNVS_tl_use:Nvv \__bnvs_gset:nnnn
739 }
740 \BNVS_new:cpn { gset:nnnv } #1 #2 #3 {
741   \BNVS_tl_use:nv {
742     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
743   }
744 }
745 \BNVS_new:cpn { gset:vvnv } #1 #2 #3 {
746   \BNVS_tl_use:nv {
747     \__bnvs_gset:vvnn { #1 } { #2 } { #3 }
748   }
749 }
750 \BNVS_new:cpn { gset:nvnn } {
751   \BNVS_tl_use:Nv \__bnvs_gset:nnnv
752 }
753 \cs_generate_variant:Nn \__bnvs_gset:nnnn { nnne }

```

__bnvs_gset:nnv __bnvs_gset:nnv {<id>} {<path>} {<key>}

Syntactic sugar for __bnvs_gset:nnnv {<id>} {<path>} {<key>} {<key>}.

```

754 \BNVS_new:cpn { gset:nnv } #1 #2 #3 {
755   \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { #3 }
756 }

```

__bnvs_gunset:nnn __bnvs_gunset:nnv {<id>} {<path>} {<key>}

__bnvs_gunset:(nvn|vvnn)

Removes the specifications for the <id>!<path>/<key> combination.

```

757 \seq_new:N \l__bnvs_I_seq
758 \BNVS_new:cpn { seq_gremove_once:Nn } #1 #2 {
759   \seq_clear:N \l__bnvs_I_seq
760   \cs_set:Npn \BNVS_seq_gremove_Nn:n ##1 {
761     \tl_if_eq:nnTF { ##1 } { #2 } {
762       \cs_set:Npn \BNVS_seq_gremove_Nn:n ####1 {
763         \seq_put_right:Nn \l__bnvs_I_seq { ####1 }
764       }
765     } {
766       \seq_put_right:Nn \l__bnvs_I_seq { ##1 }
767     }
768   }

```



```

769 \seq_map_function:NN #1 \BNVS_seq_gremove_Nn:n
770 \seq_gset_eq:NN #1 \l__bnvs_I_seq
771 }
772 \BNVS_new:cpn { gunset_IP:Nw } #1 #2 #3 {
773   \__bnvs_seq_gremove_once:Nn #1 { #3 }
774   \seq_if_empty:NT #1 {
775     \__bnvs_seq_gremove_once:Nn \g__bnvs_I_seq { #2 }
776     \cs_undefine:c { __bnvs_gset@#2:nnn}
777   }
778 }
779 \BNVS_new:cpn { gunset:NNnnn } #1 #2 #3 #4 #5 {
780   \tl_if_exist:NT #1 {
781     \cs_undefine:N #1
782     \__bnvs_seq_gremove_once:Nn #2 { #5 }
783     \seq_if_empty:NT #2 {
784       \exp_args:Nc \__bnvs_gunset_IP:Nw { \__bnvs_g_I:w { #3 } } { #3 } { #4 }
785       \cs_undefine:c { __bnvs_gset@#3!#4:nn}
786     }
787   }
788 }
789 \BNVS_new:cpn { gunset:nnn } #1 #2 #3 {
790   \exp_args:Ncc
791   \__bnvs_gunset:NNnnn
792   { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
793   { \__bnvs_g_IP:w { #1 } { #2 } }
794   { #1 } { #2 } { #3 }
795 }
796 \BNVS_new:cpn { gunset:nvn } #1 {
797   \BNVS_tl_use:nv { \__bnvs_gunset:nnn { #1 } }
798 }
799 \BNVS_new:cpn { gunset:vvv } {
800   \BNVS_tl_use:Nvv \__bnvs_gunset:nnn
801 }

```

```

\__bnvs_is_gset:nnnTF \__bnvs_is_gset:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}
\__bnvs_is_gset:(nvn|vvv|vxn)TF

```

Test for the existence of some data for that $\langle id \rangle ! \langle path \rangle / \langle key \rangle$ combination.

```

802 \BNVS_new_conditional:cpnn { is_gset:nnn } #1 #2 #3 { T, F, TF } {
803   \tl_if_exist:cTF { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
804   \prg_return_true: \prg_return_false:
805 }
806 \BNVS_new_conditional:cpnn { is_gset:vxn } #1 #2 #3 { T, F, TF } {
807   \exp_args:NNnx \BNVS_tl_use:Nv
808   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
809   \prg_return_true: \prg_return_false:
810 }
811 \BNVS_new_conditional:cpnn { is_gset:nvn } #1 #2 #3 { T, F, TF } {
812   \BNVS_tl_use:nv { \__bnvs_is_gset:nnnTF { #1 } } { #2 } { #3 }
813   \prg_return_true: \prg_return_false:
814 }

```

```

815 \BNVS_new_conditional:cpnn { is_gset:vvv } #1 #2 #3 { T, F, TF } {
816   \BNVS_tl_use:Nvv \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
817   \prg_return_true: \prg_return_false:
818 }

```

1.7.4 Loops

<code>__bnvs_foreach_I:N</code>	<code>__bnvs_foreach_I:N <function:n></code>
<code>__bnvs_foreach_I:n</code>	<code>__bnvs_foreach_I:n {<:n code>}</code>
<code>__bnvs_foreach_I_break:</code>	<code>__bnvs_foreach_I_break:</code>
<code>__bnvs_foreach_I_break:n</code>	<code>__bnvs_foreach_I_break:n {<code>}</code>

Execute the `<function:n>` or the `<:n code>` for each declared identifier. The break commands work like `\seq_map_break:` and

```

819 \BNVS_new:cpn { foreach_I:N } {
820   \seq_map_function:NN \g__bnvs_I_seq
821 }

822 \BNVS_new:cpn { foreach_I:n } {
823   \seq_map_inline:Nn \g__bnvs_I_seq
824 }

825 \cs_set_eq:NN \__bnvs_foreach_I_break: \seq_map_break:
826 \cs_set_eq:NN \__bnvs_foreach_I_break:n \seq_map_break:n

```

<code>__bnvs_foreach_P:nNTF</code>	<code>__bnvs_foreach_P:nNTF {<id>} <function:nn> {<yes:>} {<no:>}</code>
<code>__bnvs_foreach_P:nnTF</code>	<code>__bnvs_foreach_P:nnTF {<id>} {<code:nn>} {<yes:>} {<no:>}</code>

If `<id>` is a declared identifier, execute `<function:nn>` or `<code:nn>` for each combination of `<id>` and its associate `<path>`s. Both are the arguments passed to `<function:nn>` or `<:nn code>` body (through `##1` and `##2`).

```

827 \BNVS_new_conditional:cpnn { foreach_P:nN } #1 #2 { T, F, TF } {
828   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
829     \seq_map_inline:cn {
830       \__bnvs_g_I:w { #1 }
831     } {
832       #2 { #1 } { ##1 }
833     }
834     \prg_return_true:
835   } {
836     \prg_return_false:
837   }
838 }

839 \BNVS_new_conditional:cpnn { foreach_P:nn } #1 #2 { T, F, TF } {
840   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
841     \cs_set:Npn \BNVS_foreach_P_nn:nn ##1 ##2 { #2 }
842     \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
843       \BNVS_foreach_P_nn:nn { #1 } { ##1 }
844     }
845     \prg_return_true:

```

```

846 } {
847   \prg_return_false:
848 }
849 }

```

```

\__bnvs_foreach_K:nnN \__bnvs_foreach_K:nnN {<id>} {<path>} {<function:nnn>}
\__bnvs_foreach_K:nnn \__bnvs_foreach_K:nnn {<id>} {<path>} {<code:nnn>}

```

Execute the $\langle function:nnn \rangle$ or the $\langle code:nnn \rangle$ body for each of $\langle id \rangle ! \langle tag \rangle / \langle key \rangle$ combination. These are the arguments to the function or code body.

```

850 \BNVS_new:cpn { foreach_K:NnnN } #1 #2 #3 #4 {
851   \seq_if_exist:NT #1 {
852     \seq_map_inline:Nn #1 {
853       #4 { #2 } { #3 } { ##1 }
854     }
855   }
856 }

857 \BNVS_new:cpn { foreach_K:nnN } #1 #2 #3 {
858   \exp_args:Nc \__bnvs_foreach_K:NnnN
859   { \__bnvs_g_IP:w { #1 } { #2 } }
860   { #1 } { #2 } #3
861 }

862 \BNVS_new:cpn { foreach_K:nnn } #1 #2 #3 {
863   \cs_set:Npn \BNVS_foreach_K_nnn:w ##1 ##2 ##3 { #3 }
864   \__bnvs_foreach_K:nnN { #1 } { #2 } \BNVS_foreach_K_nnn:w
865 }

```

```

\__bnvs_foreach_R:nnNTF \__bnvs_foreach_R:nnNTF {<id>} {<tag>} {<function:nn>} {<yes:>} {<no:>}
\__bnvs_foreach_R:nnnTF \__bnvs_foreach_R:nnnTF {<id>} {<tag>} {<code:nn>} {<yes:>} {<no:>}

```

If the $\langle id \rangle$ is known, execute the $\langle function:nn \rangle$ or the $\langle code:nn \rangle$ body for each of $\langle id \rangle ! \langle tag \rangle . \langle subtag \rangle$ combination. The $\langle id \rangle$ and $\langle tag \rangle . \langle subtag \rangle$ are the arguments of the function and the function body. After that the $\langle yes: \rangle$ code is executed if some combination is found, otherwise it executes $\langle no: \rangle$ code.

```

866 \BNVS_new_conditional:cpnn { foreach_R:nnn } #1 #2 #3 { T, F, TF } {
867   \cs_set:Npn \BNVS_foreach_R_nnnTF:nn ##1 ##2 {
868     #3
869   }
870   \__bnvs_foreach_R:nnNTF { #1 } { #2 } \BNVS_foreach_R_nnnTF:nn
871   \prg_return_true: \prg_return_false:
872 }

873 \BNVS_new_conditional:cpnn { foreach_R:nnN } #1 #2 #3 { T, F, TF } {
874   \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {

```

This is not reentrant.

```

875   \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
876     \cs_set:Npn \BNVS_foreach_R_nnnTF_return: { \prg_return_true: }
877     \str_if_in:nnTF { ~ ##1 . } { ~ #2 . } {
878       #3 { #1 } { ##1 }
879     } {

```

```

880     }
881   }
882   \BNVS_foreach_R_nnnTF_return:
883 } {
884   \prg_return_false:
885 }
886 }

```

```

\__bnvs_foreach_IP:N \__bnvs_foreach_IP:N {function:nn}
\__bnvs_foreach_IP:n \__bnvs_foreach_IP:n {code:nn}

```

Execute the $\langle function:nn \rangle$ or the $\langle code:nn \rangle$ for each $\langle id \rangle ! \langle path \rangle$ combination.

```

887 \BNVS_new:cpn { foreach_IP:N } #1 {
888   \__bnvs_foreach_I:n {
889     \__bnvs_foreach_P:nNT { ##1 } #1 { }
890   }
891 }

892 \BNVS_new:cpn { foreach_IP:NT } #1 #2 {
893   \__bnvs_foreach_I:n {
894     \__bnvs_foreach_P:nNT { ##1 } #1 { #2 }
895   }
896 }

897 \BNVS_new:cpn { foreach_IP:n } #1 {
898   \cs_set:Npn \BNVS_foreach_IP_n:nn ##1 ##2 { #1 }
899   \__bnvs_foreach_I:n {
900     \__bnvs_foreach_P:nNT { ##1 } \BNVS_foreach_IP_n:nn { }
901   }
902 }

```

1.7.5 Path function

```

\__bnvs_walk_path:nnnn \__bnvs_walk_path:nnnn {path} {relative} {first:n} {other:n}

```

$\langle path \rangle$ is a dot separated list of components.

```

903 \BNVS_seq_new:c { walk }
904 \BNVS_set:cpn { walk_path:nnnn } #1 #2 #3 #4 {
905   \BNVS_set:cpn { walk_path_ii:n } ##1 { #2 }
906   \BNVS_set:cpn { walk_path_iii:n } ##1 { #3 }
907   \BNVS_set:cpn { walk_path_iiii:n } ##1 { #4 }
908   \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
909   \BNVS_set:cpn { walk_path_next: } { }
910   \cs_set:Npn \KWN:n ##1 {
911     \BNVS_set:cpn { walk_path_next: } {
912       \__bnvs_walk_path_ii:n { ##1 }
913     }
914     \cs_set:Npn \KWN:n ####1 {
915       \__bnvs_walk_path_iii:n { ##1 }

```

```

916     \BNVS_set:cpn { walk_path_next: } { }
917     \cs_set:Npn \KWN:n #####1 {
918         \__bnvs_walk_path_iiii:n { #####1 }
919     }
920     \__bnvs_walk_path_iiii:n { #####1 }
921 }
922 }
923 \BNVS_seq_use:Nc \seq_map_function:NN { walk } \KWN:n
924 \__bnvs_walk_path_next:
925 }

```

```

\__bnvs_foreach_relative:nn \__bnvs_foreach_relative:nn {<relative>} {<code:n>}

```

`<relative>` is a dot separated list of components starting with a dot.

```

926 \BNVS_set:cpn { foreach_relative:nn } #1 #2 {
927     \BNVS_set:cpn { foreach_relative:n } ##1 { #2 }
928     \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
929     \cs_set:Npn \KWN:n ##1 {
930         \cs_set:Npn \KWN:n { \__bnvs_foreach_relative:n }
931     }
932     \BNVS_seq_use:Nc \seq_map_function:NN { walk } \KWN:n
933 }

```

1.7.6 Low level IP model functions

1.7.7 Low level I model functions

1.7.8 Getting

```

\__bnvs_if_get:nnnTF \__bnvs_if_get:nnnTF {<id>} {<path>} {<key>} {<yes:n>} {<no:>}
\__bnvs_if_spec:nnnTF \__bnvs_if_spec:nnnTF {<id>} {<path>} {<key>} {<yes:n>} {<no:>}

```

The `\__bnvs_if_get:nnnTF` variant calls `<yes:n>` with what was stored for `<id> ! <path> / <key>`. Otherwise executes `<no:>` without changing the contents of the `<ans>` `tl` variable.

The `_spec:` variant is similar except that it uses `<id> ! <path> / <key>` combinations when available or `! <path> / <key>` otherwise.

```

934 \BNVS_new:cpn { if_get:nnnTF } #1 #2 #3 #4 #5 {
935     \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
936         \cs_set:Npn \KWN:n ##1 { #4 }
937         \exp_args:Nv \KWN:n { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
938     } {
939         #5
940     }
941 }

942 \BNVS_new:cpn { if_spec:nnnTF } #1 #2 #3 #4 #5 {
943     \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
944         \cs_set:Npn \KWN:n ##1 { #4 }
945         \exp_args:Nv \KWN:n { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
946     } {
947         \tl_if_empty:nTF { #1 } {

```

```

948     #5
949   } {
950     \__bnvs_is_gset:nnnTF {} { #2 } { #3 } {
951       \cs_set:Npn \KWN:n ##1 { #4 }
952       \exp_args:Nv \KWN:n { \__bnvs_g_IPK:w {} { #2 } { #3 } }
953     } {
954       #5
955     }
956   }
957 }
958 }

```

<code>__bnvs_if_get:nnncTF</code> <code>__bnvs_if_spec:nnncTF</code> <code>__bnvs_if_get:nncTF</code> <code>__bnvs_if_get:vvcTF</code>	<code>__bnvs_if_get:nnncTF {<id>}{<path>}{<key>}{<ans>}{<yes:>}{<no:>}</code> <code>__bnvs_if_spec:nnncTF {<id>}{<path>}{<key>}{<ans>}{<yes:>}{<no:>}</code> <code>__bnvs_if_get:nncTF {<id>}{<path>}{<ans>}{<yes:>}{<no:>}</code>
---	---

The `\__bnvs_if_get:nnnc...` variant puts what was stored for `<id>!``<path>/<key>` into the `<ans>` variable, if any, then executes `<yes:>`. Otherwise executes `<no:>` without changing the contents of the `<ans>` tl variable.

`\__bnvs_if_get:nncTF` is a convenient shortcut for `\__bnvs_if_get:nnncTF` when `<key>` and `<ans>` happen to be the same.

The `_spec:` variant is similar except that it uses `<id>!``<path>/<key>` combinations when available or `!``<path>/<key>` otherwise.

```

959 \BNVS_new_conditional:cpnn { if_get:nnnc } #1 #2 #3 #4 { T, F, TF } {
960   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
961     \BNVS_tl_use:Nc \cs_set_eq:Nc
962       { #4 } { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
963     \prg_return_true:
964   } {
965     \prg_return_false:
966   }
967 }
968 \BNVS_undefine:c { if_get:nnnc }

969 \BNVS_new_conditional:cpnn { if_spec:nnnc } #1 #2 #3 #4 { T, F, TF } {
970   \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
971     \tl_set_eq:cc { \BNVS_l:cn { #4 } { tl } } {
972       \__bnvs_g_IPK:w { #1 } { #2 } { #3 }
973     }
974     \prg_return_true:
975   } {
976     \tl_if_empty:nTF { #1 } {
977       \prg_return_false:
978     } {
979       \__bnvs_if_spec:nnncTF {} { #2 } { #3 } { #4 }
980       \prg_return_true: \prg_return_false:
981     }
982   }
983 }
984 \BNVS_undefine:c { if_spec:nnnc }

```

```

985 \BNVS_new_conditional:cpnn { if_get:nnc } #1 #2 #3 { T, F, TF } {
986   \_bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #3 }
987   \prg_return_true: \prg_return_false:
988 }

989 \BNVS_new_conditional:cpnn { if_get:vvc } #1 #2 #3 { T, F, TF } {
990   \BNVS_tl_use:Nvv \_bnvs_if_get:nncTF { #1 } { #2 } { #3 }
991   \prg_return_true: \prg_return_false:
992 }

```

_bnvs_if_resolved:nncTF _bnvs_if_resolved:nnncTF {<id>} {<path>} {<ans>} {<yes:>} {<no:>}

Execute <yes:> if resolution succeeded for either <id>!<path>/r or the fallback
!<path>/r, otherwise execute <no:>.

```

993 \BNVS_new_conditional:cpnn { if_resolved:nnc } #1 #2 #3 { T, F, TF } {
994   \_bnvs_if_get:nnncTF { #1 } { #2 } r { #3 } {
995     \prg_return_true:
996     \prg_return_false:
997   } {
998     \tl_if_empty:nTF { #1 } {
999       \prg_return_false:
1000     } {
1001       \_bnvs_if_resolved:nnncTF { } { #2 } r { #3 }
1002       \prg_return_true:
1003       \prg_return_false:
1004     }
1005   }
1006 }
1007 \BNVS_undefine:c { if_resolved:nnc }

```

_bnvs_if_resolved:nnncTF _bnvs_if_resolved:nnncTF {<id>} {<path>} {<key>} {<ans>} {<yes:>} {<no:>}

Execute <yes:> if resolution succeeded for either <id>!<path>/<key> or the fallback
!<path>/<key>, otherwise execute <no:>.

```

1008 \BNVS_new_conditional:cpnn { if_resolved:nnnc } #1 #2 #3 #4 { T, F, TF } {
1009   \_bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #4 } {
1010     \_bnvs_is_will_gset:cTF { #3 }
1011     \prg_return_true: \prg_return_false:
1012   } {
1013     \tl_if_empty:nTF { #1 } {
1014       \prg_return_false:
1015     } {
1016       \_bnvs_if_resolved:nnncTF { #1 } { #2 } { #3 } { #4 }
1017       \prg_return_true: \prg_return_false:
1018     }
1019   }
1020 }

```

_bnvs_if_resolved_IP:wTF _bnvs_if_resolved_IP:wTF {<id>} {<path>} {<yes:n>} {<no:>}

If <id>!<path>/r has been set, execute <yes:n> code with that value as argument,
otherwise execute <no:> code.

```

1021 \BNVS_tl_new:c { if_resolved_IP_wTF }
1022 \BNVS_new:cpn { if_resolved_IP:wTF } #1 #2 {
1023   \__bnvs_if_get:nnncTF { #1 } { #2 } r { if_resolved_IP_wTF } {
1024     \BNVS_tl_use:Nv \BNVS_apply_i:nnn { if_resolved_IP_wTF }
1025   } {
1026     \use_ii:nn
1027   }
1028 }

```

<code>__bnvs_require:nnnc</code> <code>__bnvs_require:nn</code> <code>__bnvs_require:vvc</code>	<code>__bnvs_require:nnnc {<id>} {<path>} {<key>} {<ans>}</code> <code>__bnvs_require:nn {<id>} {<path>} {<ans>}</code> <code>__bnvs_require:vvc</code>
--	--

Calls `\__bnvs_if_get:nnncTF`, does nothing on success and, on failure, does nothing or raise when in debug mode. `\__bnvs_require:nn` is a shortcut for the more qualified function `\__bnvs_require:nnnc` when `<key>` and `<ans>` happen to be identical.

```

1029 \BNVS_new:cpn { require:nnnc } #1 #2 #3 #4 {
1030   \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { #4 } {
1031     \BNVS_fatal_unreachable:
1032   }
1033 }

```

<code>__bnvs_gunset:nn</code> <code>__bnvs_gunset:(nv vn vv)</code>	<code>__bnvs_gunset:nn {<id>} {<path>}</code>
--	--

Removes the specifications for the `<id> ! <path>` and all possible keys combination.

```

1034 \BNVS_new:cpn { gunset:NNNnn } #1 #2 #3 #4 #5 {
1035   \seq_map_inline:Nn #1 {
1036     \__bnvs_gunset:nnn { #4 } { #5 } { ##1 }
1037   }
1038   \cs_undefine:N #1
1039   \__bnvs_seq_gremove_once:Nn #2 { #5 }
1040   \seq_if_empty:NT #2 {
1041     \cs_undefine:N #2
1042     \__bnvs_seq_gremove_once:Nn #3 { #4 }
1043     \cs_undefine:c { __bnvs_gset@ #4 :nnn }
1044   }
1045 }
1046 \BNVS_new:cpn { gunset:Nnn } #1 #2 #3 {
1047   \seq_map_inline:Nn #1 {
1048     \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
1049       \exp_args:Nc \__bnvs_gunset:NNNnn
1050         { \__bnvs_g_IP:w { #2 } { ##1 } }
1051       #1 \g__bnvs_I_seq { #2 } { ##1 }
1052       \cs_undefine:c { __bnvs_gset@{ #2 } { ##1 }:nn }
1053     }
1054   }
1055 }
1056 \BNVS_new:cpn { gunset:nn } #1 #2 {
1057   \exp_args:Nc
1058   \__bnvs_gunset:Nnn { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
1059 }

```



```

1060 \BNVS_new:cpn { gunshot:nv } #1 {
1061   \BNVS_tl_use:nv { \_bnvs_gunset:nn { #1 } }
1062 }

1063 \BNVS_new:cpn { gunshot:vn } {
1064   \BNVS_tl_use:cv { gunshot:nn }
1065 }

1066 \BNVS_new:cpn { gunshot:vv } {
1067   \BNVS_tl_use:Nvv \_bnvs_gunset:nn
1068 }

```

`\_bnvs_gunset_deep:nn`
`\_bnvs_gunset_deep:(nv|vv)`

`\_bnvs_gunset_deep:nn {⟨id⟩} {⟨path⟩}`

Removes the specifications for each existing $\langle id \rangle ! \langle path \rangle . \langle subpath \rangle$ combination.

```

1069 \BNVS_new:cpn { gunshot_deep:Nnn } #1 #2 #3 {
1070   \seq_if_exist:NT #1 {
1071     \seq_map_inline:Nn #1 {
1072       \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
1073         \_bnvs_gunset:nn { #2 } { ##1 }
1074       }
1075     }
1076   }
1077 }

1078 \BNVS_new:cpn { gunshot_deep:nn } #1 #2 {
1079   \exp_args:Nc \_bnvs_gunset_deep:Nnn
1080   { \_bnvs_g_I:w { #1 } }
1081   { #1 } { #2 }
1082 }

1083 \BNVS_new:cpn { gunshot_deep:nv } #1 {
1084   \BNVS_tl_use:nv { \_bnvs_gunset_deep:nn { #1 } }
1085 }

1086 \BNVS_new:cpn { gunshot_deep:vv } {
1087   \BNVS_tl_use:Nvv \_bnvs_gunset_deep:nn
1088 }

```

`\_bnvs_gunset:n` `\_bnvs_gunset:n {⟨id⟩}`
`\_bnvs_gunset:` `\_bnvs_gunset:`

Removes the specifications for the $\langle id \rangle$ and all possible tag combination. The second does the same for all possible $\langle id \rangle$.

```

1089 \BNVS_new:cpn { gunset:NNNn } #1 #2 #3 #4 {
1090   \cs_if_exist:NT #1 {
1091     \cs_undefine:N #1
1092     \seq_map_inline:Nn #2 {
1093       \__bnvs_gunset:nn { #4 } { ##1 }
1094     }
1095     \cs_undefine:N #2
1096     \__bnvs_seq_gremove_once:Nn #3 { #4 }
1097   }
1098 }
1099 \BNVS_new:cpn { gunset:n } #1 {
1100   \exp_args:Ncc \__bnvs_gunset:NNNn
1101   { \__bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } }
1102   \g__bnvs_I_seq { #1 }
1103 }
1104 \BNVS_new:cpn { gunset: } {
1105   \seq_map_inline:Nn \g__bnvs_I_seq {
1106     \__bnvs_gunset:n { ##1 }
1107   }
1108 }

```

__bnvs_VS_gunset: [__bnvs_VS_gunset:](#)

Removes any value for the “V” and “S” keys and any $\langle id \rangle ! \langle path \rangle$ combination.

```

1109 \BNVS_new:cpn { VS_gunset: } {
1110   \__bnvs_foreach_IP:n {
1111     \__bnvs_gunset:nnn { ##1 } { ##2 } V
1112     \__bnvs_gunset:nnn { ##1 } { ##2 } S
1113   }
1114 }

```

1.7.9 Circular dependencies

__bnvs_will_gset:nnn [__bnvs_will_gset:nnn](#) $\{ \langle id \rangle \}$ $\{ \langle path \rangle \}$ $\{ \langle key \rangle \}$

To manage circular dependencies. After this command, [__bnvs_is_will_gset:nnnTF](#) is true for the same arguments.

```

1115 \BNVS_new:cpn { will_gset:nnn } #1 #2 #3 {
1116   \__bnvs_gset:nnnn { #1 } { #2 } { #3 } { \q_no_value }
1117 }

```

```

__bnvs_is_will_gset:cTF __bnvs_is_will_gset:cTF {<id>} {<path>} {<key>}

```

To manage circular dependencies. See `__bnvs_will_gset:nnn` above.

```

1 __bnvs_will_gset:nnn {<id>} {<path>} {<key>}
2 ...
3 __bnvs_if_get:nnncT {<id>} {<path>} {<key>} {<in>} {
4   ...
5   __bnvs_is_will_gset:cTF
6     {<in>}
7     {<yes:>}
8     {<no:>}
9   ...
10 }

```

```

1118 \BNVS_new_conditional:cpnn { is_will_gset:c } #1 { T, F, TF } {
1119   \BNVS_tl_use:Nv \quark_if_no_value:nTF { #1 } {
1120     \prg_return_true:
1121   } {
1122     \prg_return_false:
1123   }
1124 }

```

1.7.10 Spring cleaning

```

__bnvs_gclear:nn __bnvs_gclear:nn {<id>} {<path>}
__bnvs_gclear:nv __bnvs_gclear:n {<id>}
__bnvs_gclear:  __bnvs_gclear:
__bnvs_greset:nn __bnvs_greset:nn {<id>} {<path>}
__bnvs_greset:nv __bnvs_greset:n {<id>}
__bnvs_greset:  __bnvs_greset:

```

The first ones clear out every data stored for `<id> ! <path>`, including deeper and registration data. When `<path>` is omitted, any known path is considered. When `<id>` is omitted, any known id is considered. The second one only clears out the data created on the fly after the initialization step. The initial data (for `key =`) is left untouched.

```

1125 \BNVS_new:cpn { greset:nn } #1 #2 {
1126   __bnvs_foreach_P:nnT { #1 } {
1127     \tl_if_in:nnT { \q_bvns ##2 . } { \q_bvns #2 . } {
1128       __bnvs_foreach_K:nnn { #1 } { ##2 } {
1129         \tl_if_in:nnF { \q_bvns ####3 } { \q_bvns = } {
1130           __bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1131         }
1132       }
1133     }
1134   } { }
1135 }

1136 \BNVS_new:cpn { greset:nv } #1 {
1137   \BNVS_tl_use:nv { __bnvs_greset:nn { #1 } }
1138 }

```

Clears out every data associated to $\langle id \rangle ! \langle path \rangle$, and any $\langle id \rangle ! \langle path \rangle . \langle subpath \rangle$.

```

1139 \BNVS_new:cpn { gclear:nn } #1 #2 {
1140   \__bnvs_foreach_P:nnT { #1 } {
1141     \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
1142       \__bnvs_foreach_K:nnn { #1 } { ##2 } {
1143         \tl_if_in:nnTF { \q_bnvs ####3 } { \q_bnvs = } {
1144           \__bnvs_gset:nnnn { #1 } { ##2 } { ####3 } { 0 }
1145         } {
1146           \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
1147         }
1148       }
1149     }
1150   } { }
1151 }

1152 \BNVS_new:cpn { gclear:n } #1 {
1153   \__bnvs_foreach_P:nNT { #1 } \__bnvs_gclear:nn {}
1154 }

1155 \BNVS_new:cpn { gclear: } {
1156   \__bnvs_foreach_IP:N \__bnvs_gclear:nn
1157 }

1158 \BNVS_new:cpn { greset:n } #1 {
1159   \__bnvs_foreach_P:nNT { #1 } \__bnvs_greset:nn {}
1160 }

1161 \BNVS_new:cpn { greset: } {
1162   \__bnvs_foreach_IP:N \__bnvs_greset:nn
1163 }

```

__bnvs_gclear_resolved:nn __bnvs_gclear_resolved:nn { $\langle id \rangle$ } { $\langle path \rangle$ }

__bnvs_gclear_resolved:nv Clear the resolved data, aka any key that does not start with “=”.

```

1164 \quark_new:N \q__bnvs_X

1165 \BNVS_new:cpn { gclear_resolved:nn } #1 #2 {
1166   \__bnvs_foreach_IP:nnn { #1 } { #2 } {
1167     \tl_if_in:nnF { \q__bnvs_X ##1 } { \q__bnvs_X = } {
1168       \__bnvs_gunset:nnn { #1 } { #2 } { ##1 }
1169     }
1170   }
1171 }

1172 \BNVS_new:cpn { gclear_resolved:nv } #1 {
1173   \BNVS_tl_use:nv {
1174     \__bnvs_gclear_resolved:nn { #1 }
1175   }
1176 }

```

1.7.11 The provide mode

`\l__bnvs_provide_bool` Manage the provide mode.

```
1177 \bool_new:N \l__bnvs_provide_bool
      (End of definition for \l__bnvs_provide_bool.)
```

```
1178 \BNVS_new:cpn { provide_ON: } {
1179   \__bnvs_set_true:c { provide }
1180 }

1181 \BNVS_new:cpn { provide_OFF: } {
1182   \__bnvs_set_false:c { provide }
1183 }
1184 \__bnvs_provide_OFF:
```

<code>__bnvs_is_gset_and_provide:nnnTF</code>	<code>__bnvs_is_gset_and_provide:nnnTF {<id>} {<path>} {<key>} {<yes:>}</code>
<code>__bnvs_is_gset_and_provide:(nvn vvn)TF</code>	<code>{<no:>}</code>
<code>__bnvs_is_gset_and_provide:nnTF</code>	<code>__bnvs_is_gset_and_provide:nnTF {<id>} {<path>} {<yes:>} {<no:>}</code>

Execute `<yes:>` when in provide mode and `<id> ! <path> / <key>` is known, `<no:>` otherwise. When not specified, `<key>` is “=”.

```
1185 \BNVS_new_conditional:cpnn { is_gset_and_provide:nnn }
1186   #1 #2 #3 { T, F, TF } {
1187     \__bnvs_if:cTF { provide } {
1188       \if_cs_exist:w \__bnvs_g_IPK:w { #1 } { #2 } { #3 } \cs_end:
1189       \prg_return_true:
1190     } \else:
1191       \prg_return_false:
1192     } \fi:
1193   } {
1194     \prg_return_false:
1195   }
1196 }

1197 \BNVS_new_conditional:cpnn { is_gset_and_provide:nvn }
1198   #1 #2 #3 { T, F, TF } {
1199     \BNVS_tl_use:nv {
1200       \__bnvs_is_gset_and_provide:nnnTF { #1 }
1201     } { #2 } { #3 }
1202     \prg_return_true: \prg_return_false:
1203   }

1204 \BNVS_new_conditional:cpnn { is_gset_and_provide:vvn }
1205   #1 #2 #3 { T, F, TF } {
1206     \BNVS_tl_use:Nvv \__bnvs_is_gset_and_provide:nnnTF { #1 } { #2 } { #3 }
1207     \prg_return_true: \prg_return_false:
1208   }
```

<code>__bnvs_gprovide:TnnnnF</code>	<code>__bnvs_gprovide:TnnnnF {<pre code>} {<id>} {<path>} {<key>} {<value>} {<no:>}</code>
--------------------------------------	---

`\__bnvs_gprovide:TvnnnF` Execute `<no:>` exclusively when not in provide mode. Does nothing when something was set for the `<id> ! <path> / <key>` combination. Execute `<pre code>` before setting.

```

1209 \BNVS_new:cpn { gprovide:TnnnnF } #1 #2 #3 #4 #5 {
1210   \__bnvs_if:cTF { provide } {
1211     \__bnvs_is_gset:nnnF { #2 } { #3 } { #4 } {
1212       #1
1213       \__bnvs_gset:nnnn { #2 } { #3 } { #4 } { #5 }
1214     }
1215   }
1216 }

1217 \BNVS_new:cpn { gprovide:TvnnnF } #1 {
1218   \BNVS_tl_use:nv { \__bnvs_gprovide:TnnnnF { #1 } }
1219 }
1220 \BNVS_new:cpn { gprovide:TvvnnF } #1 {
1221   \BNVS_tl_use:nvv { \__bnvs_gprovide:TnnnnF { #1 } }
1222 }

```

__bnvs_is_provided:nnnTF __bnvs_is_provided:nnnTF {<id>} {<path>} {<key>} {<yes:>} {<no:>}

If <id>!<path>/<key> has been successfully resolved and not unset since then, execute <yes:>. Otherwise execute <no:>.

```

1223 \tl_new:N \l__bnvs_is_provided_tl
1224 \BNVS_new_conditional:cpnn { is_provided:nnn } #1 #2 #3 { T, F, TF } {
1225   \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { is_provided } {
1226     \__bnvs_is_will_gset:cTF { is_provided } {
1227       \prg_return_false:
1228     } {
1229       \prg_return_true:
1230     }
1231   } {
1232     \prg_return_false:
1233   }
1234 }

```

1.7.12 Initialize

__bnvs_ginit:nnnn __bnvs_ginit:nnnn {<id>} {<short>} {<relative>} {<definition>}

Called by `\__bnvs_gdef:nnnn`. This function supports provide mode by calling `\__bnvs_is_gset_and_provide:nnnTF`. Somehow a shortcut to `\__bnvs_gset:nnnn` with key “=”. <relative> is a list of components, if it is not empty, each intermediate path is modified or defined appropriately by calling `\__bnvs_greset:nn` among other things.

```

1235 \BNVS_new:cpn { ginit:nnnn } #1 #2 #3 #4 {
1236   \tl_if_empty:nTF { #3 } {
1237     \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
1238       \__bnvs_gunset_deep:nn { #1 } { #2 }
1239       \__bnvs_gset:nnnn { #1 } { #2 } = { #4 }
1240     }
1241   } {
1242     \BNVS_begin:

```

```

1243 \__bnvs_greset:nn { #1 } { #2 }
1244 \__bnvs_is_gset:nnnF { #1 } { #2 } = {
1245   \__bnvs_gset:nnnn { #1 } { #2 } = {}
1246 }
1247 \__bnvs_tl_set:cn P { #2 }
1248 \__bnvs_foreach_relative:nn { #3 } {
1249   \__bnvs_tl_put_right:cn P { . ##1 }
1250   \__bnvs_greset:nv { #1 } P
1251   \__bnvs_is_gset:nvnF { #1 } P = {

```

Each intermediate step is just defined empty when not already defined. The last step setting is overridden just after the loop.

```

1252   \__bnvs_gset:nvnn { #1 } P = {}
1253 }
1254 }
1255 \__bnvs_is_gset_and_provide:nvnF { #1 } P = {
1256   \__bnvs_gunset_deep:nv { #1 } P
1257   \__bnvs_gset:nvnn { #1 } P = { #4 }
1258 }
1259 \BNVS_end:
1260 }
1261 }

```

```

\__bnvs_ginit:nnn \__bnvs_ginit:nnn {<id>} {<path>} {<simple definition>}

```

```

\__bnvs_ginit:(nnv|vvn)

```

Called by `\__bnvs_gdef:nnn`. Somehow a shortcut to `\__bnvs_gset:nnnn` with key “=”. If `<path>` has more than one component, each intermediate path is modified or defined appropriately. This function supports provide mode by calling `\__bnvs_is_gset_and_provide:nnnTF`.

```

1262 \BNVS_new:cpn { ginit:nnn } #1 #2 {
1263   \__bnvs_ginit:nnnn { #1 } { #2 } {}
1264 }

1265 \BNVS_new:cpn { ginit:nnv } #1 #2 {
1266   \BNVS_tl_use:nv { \__bnvs_ginit:nnn { #1 } { #2 } }
1267 }

1268 \BNVS_new:cpn { ginit:vvn } {
1269   \BNVS_tl_use:Nvv \__bnvs_ginit:nnn
1270 }

```

```

\__bnvs_if_should_ginit:nnTF \__bnvs_if_should_ginit:nnTF {<id>} {<path>} {<yes:>} {<no:>}

```

Execute `<yes:>` when in provide mode and `<id>!<tag>` combination is not initialized by `\__bnvs_ginit:nnn`. Execute `<no:>` otherwise.

```

1271 \BNVS_new_conditional:cpnn { if_should_ginit:nn } #1 #2 { T, F, TF } {
1272   \__bnvs_if:cTF { provide } {
1273     \__bnvs_is_ginit:nnTF { #1 } { #2 } {
1274       \prg_return_false:
1275     } {
1276       \prg_return_true:
1277     }
1278   } {
1279     \prg_return_true:

```

```

1280 }
1281 }
1282 \BNVS_undefine:c { if_should_ginit:nn }

```

`\__bnvs_is_ginit:nnTF` `\__bnvs_is_ginit:nnTF {⟨id⟩} {⟨path⟩} {⟨yes:⟩} {⟨no:⟩}`

Test for the existence of some data for that $\langle id \rangle ! \langle path \rangle$ combination.

```

1283 \BNVS_new_conditional:cpnn { is_ginit:nn } #1 #2 { T, F, TF } {
1284   \__bnvs_is_gset:nnTF { #1 } { #2 } = \prg_return_true: \prg_return_false:
1285 }
1286 \BNVS_undefine:c { is_ginit:nn }

```

`\__bnvs_if_ginit:nncTF` `\__bnvs_if_ginit:nncTF {⟨id⟩} {⟨tag⟩} {⟨var⟩} {⟨yes:⟩} {⟨no:⟩}`

Test for the existence of some initialization data for that $\langle id \rangle ! \langle path \rangle$ combination. On success, $\langle var \rangle$ is set to this data.

```

1287 \BNVS_new_conditional:cpnn { if_ginit:nnc } #1 #2 #3 { T, F, TF } {
1288   \__bnvs_if_get:nnncTF { #1 } { #2 } = { #3 } \prg_return_true: \prg_return_false:
1289 }
1290 \BNVS_undefine:c { if_ginit:nnc }

```

1.8 Regular expressions

`\c__bnvs_A_reserved_Z_regex` Reserved words.

(End of definition for `\c__bnvs_A_reserved_Z_regex`.)

```

1291 \regex_const:Nn \c__bnvs_A_reserved_Z_regex {
1292   \A (? : _+ \d | _* [a-z] ) [_a-z0-9]* \Z
1293 }

```

`\c__bnvs_A_VAZLLZZL_Z_regex` Used to parse named overlay specifications. V, A:Z, A::L on one side, :Z, :Z::L and ::L:Z on the other sides. Next are the capture groups. The first one is for the whole match.

(End of definition for `\c__bnvs_A_VAZLLZZL_Z_regex`.)

```

1294 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
1295   \A \s* (? :

```

- 2 $\rightarrow V$

```

1296     ( [^:]+? )

```

- 3, 4, 5 $\rightarrow A : Z?$ or $A :: L?$

```

1297     | (? : ( [^:]+? ) \s* : (? : \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )

```

- 6, 7 $\rightarrow ::(L:Z)?$

```

1298     | (? : :: \s* (? : ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )

```

- 8, 9 $\rightarrow :(Z::L)?$

```

1299     | (? : : \s* (? : ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
1300   )
1301   \s* \Z
1302 }

```


1.9 End group setters

```
1303 \cs_new:Npn \BNVS_after_end_tl_put_right:cv #1 {
1304   \BNVS_after_end:n {
1305     \__bnvs_tl_put_right:cn { #1 }
1306   }
1307   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1308 }

1309 \cs_new:Npn \BNVS_end_tl_put_right:cv #1 #2 {
1310   \BNVS_after_end_tl_put_right:cv { #1 } { #2 }
1311   \BNVS_end:
1312 }

1313 \cs_new:Npn \BNVS_after_end_tl_gset:nnnv #1 #2 #3 {
1314   \BNVS_after_end:n {
1315     \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
1316   }
1317   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1318 }

1319 \cs_new:Npn \BNVS_end_tl_gset:nnnv #1 #2 #3 #4 {
1320   \BNVS_after_end_tl_gset:nnnv { #1 } { #2 } { #3 } { #4 }
1321   \BNVS_end:
1322 }

1323 \cs_new:Npn \BNVS_after_end_tl_set:cv #1 {
1324   \BNVS_after_end:n {
1325     \__bnvs_tl_set:cn { #1 }
1326   }
1327   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1328 }

1329 \cs_new:Npn \BNVS_end_tl_set:cv {
1330   \BNVS_tl_set:ncv { \BNVS_end: }
1331 }

1332 \cs_new:Npn \BNVS_tl_set:ncv #1 #2 {
1333   \BNVS_tl_use:nv {
1334     #1
1335     \__bnvs_tl_set:cn { #2 }
1336   }
1337 }

1338 \cs_new:Npn \BNVS_tl_set:ncvcv #1 #2 #3 #4 {
1339   \BNVS_tl_set:ncv {
1340     \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1341   } { #4 }
1342 }

1343 \cs_new:Npn \BNVS_tl_set:ncvcvcv #1 #2 #3 #4 #5 #6 {
1344   \BNVS_tl_set:ncv {
1345     \BNVS_tl_set:ncv {
1346       \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
1347     } { #4 } { #5 }
1348   } { #6 }
1349 }
```

```

1350 \cs_new:Npn \BNVS_after_end_int_set:cv #1 {
1351   \BNVS_after_end:n {
1352     \__bnvs_int_set:cn { #1 }
1353   }
1354   \BNVS_tl_use:Nv \BNVS_after_end_braced:n
1355 }

1356 \cs_new:Npn \BNVS_end_int_set:cv #1 #2 {
1357   \BNVS_after_end_int_set:cv { #1 } { #2 }
1358   \BNVS_end:
1359 }

```

1.10 beamer.cls interface

Work in progress.

```

1360 \RequirePackage{keyval}

1361 \define@key{beamerframe}{beanoves~id}[]{}
1362 \tl_set:Nx \l__bnvs_J_tl { #1 }
1363 }

1364 \bool_new:N \l__bnvs_in_frame_bool
1365 \bool_set_false:N \l__bnvs_in_frame_bool
1366 \AddToHook{env/beamer@frameslide/before}{
1367   \__bnvs_greset_call:
1368   \__bnvs_VS_gunset:
1369   \__bnvs_set_true:c { in_frame }
1370   \__bnvs_tl_clear:c I
1371   \__bnvs_tl_clear:c { S() }
1372 }

1373 \AddToHook{env/beamer@frameslide/after}{
1374   \__bnvs_set_false:c { in_frame }
1375 }

```

1.11 Utilities

1.11.1 Split utilities

```

\__bnvs_if_regex_split:cncTF \__bnvs_if_regex_split:cncTF {<regex core>} {<expression>} <seq core> {<yes:>}
\__bnvs_if_regex_split:cncTF {<no:>}
\__bnvs_if_regex_split:cncTF {<regex core>} {<expression>} {<yes:>} {<no:>}

```

These are shortcuts to `\regex_split:NnNTF` with the `split` sequence as last `N` argument.

```

1376 \BNVS_new_conditional:cpnn { if_regex_split:cnc } #1 #2 #3 { T, F, TF } {
1377   \BNVS_seq_use:nc {
1378     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1379   } { #3 } \prg_return_true: \prg_return_false:
1380 }

```

```

1381 \BNVS_new_conditional:cpnn { if_regex_split:cn } #1 #2 { T, F, TF } {
1382   \BNVS_seq_use:nc {
1383     \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
1384   } { split } \prg_return_true: \prg_return_false:
1385 }

```

<pre> __bnvs_split_pop: __bnvs_split_pop:c __bnvs_split_pop:cc __bnvs_split_pop:ccc __bnvs_split_pop:cccc __bnvs_split_pop:ccccc </pre>	<pre> __bnvs_split_pop: __bnvs_split_pop:c {<var₁>} __bnvs_split_pop:cc {<var₁>} {<var₂>} __bnvs_split_pop:ccc {<var₁>} {<var₂>} {<var₃>} __bnvs_split_pop:cccc {<var₁>} {<var₂>} {<var₃>} {<var₄>} __bnvs_split_pop:ccccc {<var₁>} {<var₂>} {<var₃>} {<var₄>} {<var₅>} </pre>
---	--

```

1386 \tl_new:N \l__bnvs_split_pop_tl
1387 \BNVS_new:cpn { split_pop: } {
1388   \seq_pop_left:NN \l__bnvs_split_seq \l__bnvs_split_pop_tl
1389 }

```

```

1390 \BNVS_new:cpn { split_pop:c } #1 {
1391   \BNVS_tl_use:nc {
1392     \seq_pop_left:NN \l__bnvs_split_seq
1393   } { #1 }
1394 }

```

```

1395 \BNVS_new:cpn { split_pop:cc } #1 {
1396   __bnvs_split_pop:c { #1 }
1397   \__bnvs_split_pop:c
1398 }

```

```

1399 \BNVS_new:cpn { split_pop:ccc } #1 {
1400   __bnvs_split_pop:c { #1 }
1401   __bnvs_split_pop:cc
1402 }

```

```

1403 \BNVS_new:cpn { split_pop:cccc } #1 {
1404   __bnvs_split_pop:c { #1 }
1405   __bnvs_split_pop:ccc
1406 }

```

```

1407 \BNVS_new:cpn { split_pop:ccccc } #1 {
1408   __bnvs_split_pop:c { #1 }
1409   __bnvs_split_pop:cccc
1410 }

```

```

1411 \BNVS_new_conditional:cpnn { split_if_pop:c } #1 { T, F, TF } {
1412   __bnvs_seq_pop_left:ccTF { split } { #1 } {
1413     \prg_return_true:
1414   } {
1415     \prg_return_false:
1416   }
1417 }

```

1.11.2 Match utilities

```

\__bnvs_match_if_once:NnTF \__bnvs_match_if_once:NnTF <regex variable> {<expression>} {<yes:>} {<no:>}
\__bnvs_match_if_once:NvTF \__bnvs_match_if_once:nnTF {<regex>} {<expression>} {<yes:>} {<no:>}
\__bnvs_match_if_once:nnTF

```

These are shortcuts to

- \regex_match_if_once:NnNTF with the match sequence as N argument
- \regex_match_if_once:nnNTF with the match sequence as N argument
- \regex_split:NnNTF with the split sequence as last N argument

```

1418 \BNVS_new_conditional:cpnn { if_extract_once:Ncn } #1 #2 #3 { T, F, TF } {
1419   \BNVS_use:ncn {
1420     \regex_extract_once:NnNTF #1 { #3 }
1421   } { #2 } { seq } \prg_return_true: \prg_return_false:
1422 }

```

\l__bnvs_match_seq Local storage for the match result.

(End of definition for \l__bnvs_match_seq.)

```

1423 \BNVS_new_conditional:cpnn { match_if_once:Nn } #1 #2 { T, F, TF } {
1424   \BNVS_use:ncn {
1425     \regex_extract_once:NnNTF #1 { #2 }
1426   } { match } { seq } \prg_return_true: \prg_return_false:
1427 }

1428 \BNVS_new_conditional:cpnn { if_extract_once:Ncv } #1 #2 #3 { T, F, TF } {
1429   \BNVS_seq_use:nc {
1430     \BNVS_tl_use:nv {
1431       \regex_extract_once:NnNTF #1
1432     } { #3 }
1433   } { #2 } \prg_return_true: \prg_return_false:
1434 }

1435 \BNVS_new_conditional:cpnn { match_if_once:Nv } #1 #2 { T, F, TF } {
1436   \BNVS_seq_use:nc {
1437     \BNVS_tl_use:nv {
1438       \regex_extract_once:NnNTF #1
1439     } { #2 }
1440   } { match } \prg_return_true: \prg_return_false:
1441 }

1442 \BNVS_new_conditional:cpnn { match_if_once:nn } #1 #2 { T, F, TF } {
1443   \BNVS_seq_use:nc {
1444     \regex_extract_once:nnNTF { #1 } { #2 }
1445   } { match } \prg_return_true: \prg_return_false:
1446 }

1447 \BNVS_new_conditional:cpnn { match_if_pop_left:c } #1 { T, F, TF } {
1448   \BNVS_tl_use:nc {
1449     \BNVS_seq_use:Nc \seq_pop_left:NNTF { match }
1450   } { #1 } {

```

```

1451     \prg_return_true:
1452   } {
1453     \prg_return_false:
1454   }
1455 }

```

```

\__bnvs_match_pop:      \__bnvs_match_pop:
\__bnvs_match_pop:c      \__bnvs_match_pop:c      {<var1>}
\__bnvs_match_pop:cc      \__bnvs_match_pop:cc      {<var1>} {<var2>}
\__bnvs_match_pop:ccc      \__bnvs_match_pop:ccc      {<var1>} {<var2>} {<var3>}
\__bnvs_match_pop:cccc      \__bnvs_match_pop:cccc      {<var1>} {<var2>} {<var3>} {<var4>}
\__bnvs_match_pop:ccccc      \__bnvs_match_pop:ccccc      {<var1>} {<var2>} {<var3>} {<var4>} {<var5>}

```

```

1456 \tl_new:N \l__bnvs_match_pop_tl
1457 \BNVS_new:cpn { match_pop: } {
1458   \seq_pop_left:NN \l__bnvs_match_seq \l__bnvs_match_pop_tl
1459 }

1460 \BNVS_new:cpn { match_pop:c } #1 {
1461   \BNVS_tl_use:nc {
1462     \seq_pop_left:NN \l__bnvs_match_seq
1463   } { #1 }
1464 }

1465 \BNVS_new:cpn { match_pop:cc } #1 {
1466   \__bnvs_match_pop:c { #1 }
1467   \__bnvs_match_pop:c
1468 }

1469 \BNVS_new:cpn { match_pop:ccc } #1 {
1470   \__bnvs_match_pop:c { #1 }
1471   \__bnvs_match_pop:cc
1472 }

1473 \BNVS_new:cpn { match_pop:cccc } #1 {
1474   \__bnvs_match_pop:c { #1 }
1475   \__bnvs_match_pop:ccc
1476 }

1477 \BNVS_new:cpn { match_pop:ccccc } #1 {
1478   \__bnvs_match_pop:c { #1 }
1479   \__bnvs_match_pop:cccc
1480 }

```

1.12 Main scanner

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual “\”, “{” and “}” before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use $\TeX$ groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

The scanning process reads $\langle content \rangle$ from the $\langle input\ stream \rangle$ and store a corresponding *abstract syntax tree* (AST) denoted by $\langle content \rangle^*$. The AST will be executed either to define named overlay sets or to resolve named overlay queries. The workflow consists of

1. begin with `\BNVS_begin_scan:`
2. define the AST exportation policy
3. define the scanning policy
4. define the branch functions named `\BNVS_scanned_⟨branch⟩`, where $\langle branch \rangle$ is one of `Block:w`, `One_colon:w`, `Colons:w`, `comMa:w`, `cClose:w`, `Stop:`,
5. call `\BNVS_scan:w` or some `\BNVS_scan_...:...w` function to peek characters from the $\langle input\ stream \rangle$ and store corresponding AST
6. end with `\BNVS_end_scan:`, which exports the AST created according to the exportation policy.

We use the same scanner to parse definitions and queries. We do not use any key–value facility built into $\LaTeX$.

1.12.1 Base grammar

Hereafter is a pragmatic grammar used for parsing, not all the expressions covered do have a real meaning. The characters all have category other. We start with basic entities.

1	$\langle spc \rangle$	::= Horizontal white space characters
2	$\langle lower \rangle$	::= a lowercase letter
3	$\langle upper \rangle$	::= an uppercase letter
4	$\langle digit \rangle$	::= a decimal digit
5	$\langle sign \rangle$	::= '-' '+'
6	$\langle comma \rangle$	::= $\langle spc \rangle$ ',' $\langle spc \rangle$
7	$\langle alpha \rangle$	::= $\langle lower \rangle$ $\langle upper \rangle$ '_'
8	$\langle alnum \rangle$	::= $\langle alpha \rangle$ $\langle digit \rangle$
9	$\langle index \rangle$	::= $\langle sign \rangle^* \langle digit \rangle^+$
10	$\langle number \rangle$	::= $\langle sign \rangle^? \langle significand \rangle \langle exponent \rangle^?$
11	$\langle significand \rangle$	::= ('.' $\langle digit \rangle^+$) ($\langle digit \rangle^+ \cdot \langle digit \rangle^*$)
12	$\langle exponent \rangle$	::= ('e' 'E') $\langle sign \rangle^? \langle digit \rangle^+$
13	$\langle other \rangle$	::= $\langle number \rangle$ '+' '-' '*' '/'

`\c__bnvs_N_regex` Signed integer.

(End of definition for `\c__bnvs_N_regex`.)

```

1481 \regex_const:Nn \c__bnvs_N_regex {
1482   [-+]* \d+
1483 }

```

`\c__bnvs_dot_N_regex` Signed integer with a dot . just before.

(End of definition for \c__bnvs_dot_N_regex.)

```

1484 \regex_const:Nn \c__bnvs_dot_N_regex {
1485   \. \ur{c__bnvs_N_regex}
1486 }

```

`\c__bnvs_A_dot_N_Z_regex` Full signed integer with a dot . jus before.

(End of definition for \c__bnvs_A_dot_N_Z_regex.)

```

1487 \regex_const:Nn \c__bnvs_A_dot_N_Z_regex {
1488   \A \ur { c__bnvs_dot_N_regex } \Z
1489 }

```

`\c__bnvs_A_i_Z_regex` Signed integer with no capture groups.

(End of definition for \c__bnvs_A_i_Z_regex.)

```

1490 \regex_const:Nn \c__bnvs_A_i_Z_regex { \A \ur{c__bnvs_N_regex} \Z }

```

`\c__bnvs_dot_R_regex` A possibly empty list of . *<short name>* items representing a path. Integers are allowed as well.

```

1491 \regex_const:Nn \c__bnvs_R_regex {
1492   \ur{c__bnvs_N_regex} | [[:alnum:]]+
1493 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_dot_R_regex` A possibly empty list of . *<short name>* items representing a path. Integers are allowed as well.

```

1494 \regex_const:Nn \c__bnvs_dot_R_regex {
1495   \. \ur{c__bnvs_R_regex}
1496 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_S_regex` This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit.

```

1497 \regex_const:Nn \c__bnvs_S_regex {
1498   [[:alpha:]] [[:alnum:]]*
1499 }

```

(End of definition for \c__bnvs_S_regex.)

`\c__bnvs_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

1500 \regex_const:Nn \c__bnvs_P_regex {
1501   \d*[:alpha:][_][[:alnum:]]_*
1502 }

```

(End of definition for `\c__bnvs_P_regex`.)

`\c__bnvs_dot_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

1503 \regex_const:Nn \c__bnvs_dot_P_regex {
1504   \. \ur{c__bnvs_P_regex}
1505 }

```

(End of definition for `\c__bnvs_dot_P_regex`.)

1.12.2 Storage

<code>\BNVS_xprt_ast:n</code>	<code>\BNVS_xprt_ast:n {⟨content⟩*}</code>
<code>\BNVS_xprt_clear:</code>	<code>\BNVS_xprt_clear:</code>

`\BNVS_xprt_ast:n` is a utility function to store its argument in the `xprted` variable. `\BNVS_xprt_clear:` resets the scan storage, called when starting a new $\text{\TeX}$ group dedicated to scanning.

```

1506 \cs_new:Npn \BNVS_xprt_clear: {
1507   \__bnvs_tl_clear:c { xprted }
1508 }

1509 \cs_new:Npn \BNVS_xprt_ast:n #1 {
1510   \BNVS_xprt_ast_Raw:
1511   \__bnvs_tl_put_right:cn { xprted } { #1 }
1512 }

```

There is a special situation for raw material that is stored separately and exported at once before other material is exported or when a scan group ends.

<code>\BNVS_xprt_Raw:n</code>	<code>\BNVS_xprt_Raw:n {⟨something⟩}</code>
<code>\BNVS_xprt_ast_Raw:</code>	<code>\BNVS_xprt_ast_Raw:</code>

`\BNVS_xprt_Raw:n` synchronously cumulates its argument in the `xprted_Raw` variable. The function `\BNVS_xprt_ast_Raw:` is executed both at the beginning of the function `\BNVS_xprt_ast:n` and when a scan group ends. It exports the contents of the stored `⟨material⟩`, when non empty, to the `xprted` variable as `\:n{⟨material⟩}` instruction and cleans the `xprted_Raw` variable.

```

1513 \cs_new:Npn \BNVS_xprt_Raw:n #1 {
1514   \tl_if_empty:nF { #1 } {
1515     \__bnvs_tl_put_right:cn { xprted_Raw } { #1 }
1516   }
1517 }

1518 \cs_new:Npn \BNVS_xprt_ast_Raw: {
1519   \__bnvs_tl_if_empty:cF { xprted_Raw } {

```



```

1520     \BNVS_tl_use:nv {
1521         \__bnvs_tl_clear:c { xprted_Raw }
1522         \BNVS_xprt_ast:n { \:n }
1523         \BNVS_xprt_grp:n
1524     } { xprted_Raw }
1525 }
1526 }

```

\BNVS_xprt_grp:n **\BNVS_xprt_grp:n** {<scanned>}

Feeds the scan storage with <scanned>, enclosed between one pair of curly braces.

```

1527 \cs_new:Npn \BNVS_xprt_grp:n #1 {
1528     \BNVS_xprt_ast:n { { #1 } }
1529 }

```

\BNVS_xprt_space: **\BNVS_xprt_space:**
\BNVS_xprt_space_set:n **\BNVS_xprt_space_set:n** {<code>}

\BNVS_xprt_space: exports the space character. **\BNVS_xprt_space_set:n** sets the meaning of **\BNVS_xprt_space:** to <code>.

```

1530 \cs_new:Npn \BNVS_xprt_space_set:n #1 {
1531     \cs_set:Npn \BNVS_xprt_space: {
1532         #1
1533     }
1534 }

1535 \BNVS_xprt_space_set:n {\BNVS_xprt_Raw:n { ~ } }

```

\BNVS_xprted_use:n **\BNVS_xprted_use:n** {<code:n>}

Puts back to the input stream the instructions <code:n> followed by the exported material so far, enclosed between one pair of braces.

```

1536 \cs_new:Npn \BNVS_xprted_use:n #1 {
1537     \BNVS_xprt_ast_Raw:
1538     \BNVS_tl_use:nv { #1 } { xprted }
1539 }

```

\BNVS_if_xprted_empty:TF **\BNVS_if_xprted_empty:TF** {<yes:n>} {<no:n>}

Execute <yes:n> instructions if something has been exported, <no:n> instructions otherwise.

```

1540 \cs_new:Npn \BNVS_if_xprted_empty:TF {
1541     \BNVS_xprted_use:n { \tl_if_empty:nTF }
1542 }

```

\BNVS_next_set:n **\BNVS_next_set:n** {<next>}
\BNVS_next_use:n **\BNVS_next_use:n** {<code>}

\BNVS_next_set:n stores <next> to be executed later by **\BNVS_next_use:n**, just after <code>. Typically, **\BNVS_next_set:n** is used in a scan group closed by <code>. **\BNVS_end_scan_next:** ends the scan and executes <next> code.

```

1543 \BNVS_tl_new:c { next }
1544 \cs_new:Npn \BNVS_next_set:n #1 {
1545   \__bnvs_tl_set:cn { next } {
1546     \exp_not:n {
1547       #1
1548     }
1549   }
1550 }

1551 \cs_new:Npn \BNVS_next_use:n #1 {
1552   \BNVS_tl_use_unbraced:nv { #1 } { next }
1553 }

```

1.12.3 AST exportation policy

This is used by `\BNVS_end_scan:` to wrap the AST created during the scanning process before it is exported. The case of an empty created AST is sometimes special.

```
\BNVS_ast_prefix_set:n \BNVS_ast_prefix_set:n {<prefix:w>}
```

Set the function `\BNVS_ast_prefix:w` to expand to `\exp_not:n {<prefix:w>}`. This is used to define `\BNVS_ast_plc:n`, when the arguments must be scanned prior to knowing which function `<prefix:w>` fits.

```

1554 \cs_new:Npn \BNVS_ast_prefix:w {}

1555 \cs_new:Npn \BNVS_ast_prefix_set:n #1 {
1556   \cs_set:Npn \BNVS_ast_prefix:w { \exp_not:n { #1 } }
1557 }

```

<code>\BNVS_ast_plc_wrap:n</code>	<code>\BNVS_ast_plc_wrap:n {<template:n>}</code>
<code>\BNVS_ast_plc_wrap_pfx:n</code>	<code>\BNVS_ast_plc_wrap_pfx:n {<template:n>}</code>
<code>\BNVS_ast_plc_wrap_mpt:n</code>	<code>\BNVS_ast_plc_wrap_grp:n {<template:n>}</code>
<code>\BNVS_ast_plc_wrap_grp:n</code>	<code>\BNVS_ast_plc_wrap_mpt:n {<template:n>}</code>
<code>\BNVS_ast_plc_Raw:</code>	<code>\BNVS_ast_plc_Raw</code>

Set the AST exportation policy through the meaning of the function `\BNVS_ast_plc:n`. The `<template:n>` is by default `##1`. With the first variant, nothing is exported if the input AST is blank. With the `pfx` variant, only the `<prefix:w>` is exported when the input AST is blank. The `mpt` and `grp` variants never export an empty AST. The first one exports the input AST as is whereas the second one exports the input AST between a pair of braces. `\BNVS_ast_plc_Raw` exports as is, with no brace nor prefix.

```

1558 \cs_new:Npn \BNVS_ast_plc_wrap:n #1 {
1559   \cs_set:Npn \BNVS_ast_plc:n ##1 {
1560     \tl_if_blank:nF { ##1 } {
1561       \BNVS_ast_prefix:w
❗ and \exp_not:n{<xprt:n>}.
1562     }
1563   }
1564 }
1565 }
1566 \BNVS_ast_plc_wrap:n { #1 }

```

```

1567 \cs_new:Npn \BNVS_ast_plc_wrap_pfx:n #1 {
1568   \cs_set:Npn \BNVS_ast_plc:n ##1 {
1569     \BNVS_ast_prefix:w
1570     \tl_if_blank:nF { ##1 } {
❧ and \exp_not:n{⟨xprt:n⟩}.
1571     \exp_not:n { #1 }
1572   }
1573 }
1574 }

1575 \cs_new:Npn \BNVS_ast_plc_wrap_mpt:n #1 {
1576   s
1577   \cs_set:Npn \BNVS_ast_plc:n ##1 {
1578     \BNVS_ast_prefix:w
❧ and \exp_not:n{⟨xprt:n⟩}.
1579     \exp_not:n { #1 }
1580   }
1581 }

1582 \cs_new:Npn \BNVS_ast_plc_wrap_grp:n #1 {
1583   s
1584   \cs_set:Npn \BNVS_ast_plc:n ##1 {
1585     \BNVS_ast_prefix:w
❧ and \exp_not:n{⟨xprt:n⟩}.
1586     \exp_not:n { { #1 } }
1587   }
1588 }

1589 \cs_new:Npn \BNVS_ast_plc_Raw: {
1590   s
1591   \cs_set:Npn \BNVS_ast_plc:n ##1 { \exp_not:n { ##1 } }
1592 }

```

1.12.4 Branch functions

The scanning process only applies to characters of category codes space and other. It definitely stops at the first token that is not of that category. It takes one of 6 branches depending on what is scanned:

- $\langle \textit{Block:w} \rangle$ for unit blocks like names,
- $\langle \textit{One_colon:w} \rangle$ after a single colon,
- $\langle \textit{Colons:w} \rangle$ after multiple colons,
- $\langle \textit{comMa:w} \rangle$ after a comma,
- $\langle \textit{cLose:w} \rangle$ after a closing delimiter,
- $\langle \textit{Stop:} \rangle$ when scanning is done.

Block branch The Block branch is used when the last character scanned from the $\langle input\ stream \rangle$ fits the current requirement as defined by the last scan function called, but the next one does not.

```

\BNVS_scan_Block:Fw      \BNVS_scan_Block:Fw { $\langle else:w \rangle$ }  $\langle input\ stream \rangle$ 
\BNVS_scanned_Block:w   \BNVS_scanned_Block:w  $\langle input\ stream \rangle$ 
\BNVS_scanned_Block_set:n \BNVS_scanned_Block_set:n { $\langle Block:w \rangle$ }
\BNVS_scanned_Block_wrap::n \BNVS_scanned_Block_wrap::n { $\langle scAn:B:w \rangle$ }
\BNVS_scanned_Block_wrap:B \BNVS_scan_Block:TFw { $\langle yes:w \rangle$ } { $\langle else:w \rangle$ }  $\langle input\ stream \rangle$ 
\BNVS_scan_Block:TFw     \BNVS_scan_Block_def:n { $\langle block:TFw \rangle$ }
\BNVS_scan_Block_def:n

```

The `\BNVS_scanned_Block:w` branch function is called by `\BNVS_scan_Block:Fw` once a $\langle block \rangle$ has been scanned from the head of the $\langle input\ stream \rangle$ (greedy match). The function `\BNVS_scanned_Block_set:n` sets the meaning of `\BNVS_scanned_Block:w` to $\langle Block:w \rangle$. The exact definition of a block is driven by `\BNVS_scan_Block:TFw` which meaning is set by `\BNVS_scan_Block_def:n`.

The `\BNVS_scanned_Block_wrap::n` is a wrapper over `\BNVS_scanned_Block_set:n`. Its argument is the body of a function with one argument which is the meaning of current `\BNVS_scanned_Block:w` at the time of the call.

```

1593 \cs_new:Npn \BNVS_scan_Block:Fw #1 {
1594   \BNVS_scan_Block:TFw {
1595     \BNVS_scanned_Block:w
1596   } {
1597     #1
1598   }

```

⊘ There must be absolutely no code here. No hooking allowed either.

```

1599 }

1600 \cs_new:Npn \BNVS_scan_Block_def:n #1 {
1601   \cs_set:Npn \BNVS_scan_Block:TFw ##1 ##2 {
1602     #1
1603   }
1604 }

1605 \cs_new:Npn \BNVS_scanned_Block_wrap_do:n #1 {
1606   \BNVS_scanned_Block_set:n {
1607     \BNVS_scanned_Block_wrapper:B { #1 }
1608   }
1609 }

1610 \cs_new:Npn \BNVS_scanned_Block_wrap::n #1 {
1611   \cs_set:Npn \BNVS_scanned_Block_wrapper:B ##1 { #1 }
1612   \exp_args:No \BNVS_scanned_Block_wrap_do:n { \BNVS_scanned_Block:w }
1613 }

1614 \cs_new:Npn \BNVS_scanned_Block_wrap::B #1 {
1615   \BNVS_scanned_Block_set:n {
1616     \BNVS_end_scan_no_I:
1617     \cs_set:Npn \BNVS_scan:B ####1 { #1 }
1618     \BNVS_scan:B { \BNVS_scanned_Block:w }
1619   }
1620 }

```

```

1621 \BNVS_scan_Block_def:n {
❗ At the top level, the  $\langle \text{else:w} \rangle$  instructions are always executed.
1622     #2
1623 }

```

Colon branch

`\c__bnvs_Colons_regex` For ranges defined by a colon syntax. One catching group for more than one colon.

```

1624 \regex_const:Nn \c__bnvs_Colons_regex { \s* :(:*) \s* }

(End of definition for \c__bnvs_Colons_regex.)

```

<code>\BNVS_scan_colon:Fw</code>	<code>\BNVS_scan_colon:Fw {$\langle \text{else:w} \rangle$} $\langle \text{input stream} \rangle$</code>
<code>\BNVS_scanned_One_colon:w</code>	<code>\BNVS_scanned_One_colon:w $\langle \text{input stream} \rangle$</code>
<code>\BNVS_scanned_One_colon_set:n</code>	<code>\BNVS_scanned_One_colon_set:n {$\langle \text{One_colon:w} \rangle$}</code>
<code>\BNVS_scanned_Colons:w</code>	<code>\BNVS_scanned_Colons:w $\langle \text{input stream} \rangle$</code>
<code>\BNVS_scanned_Colons_set:n</code>	<code>\BNVS_scanned_Colons_set:n {$\langle \text{Colons:w} \rangle$}</code>

On scanning a standalone colon from the $\langle \text{input stream} \rangle$, calls the branch function `\BNVS_scanned_One_colon:w`. On scanning multiple colons from the $\langle \text{input stream} \rangle$, calls the `\BNVS_scanned_Colons:w` branch function. In other cases, execute the $\langle \text{else:w} \rangle$ instructions.

❗ One shot utility function only used by `\BNVS_scan_colon:Fw`

```

1625 \cs_new:Npn \BNVS_scanned_Colons_regex:n #1 {
1626     \tl_if_empty:nTF { #1 } {
1627         \BNVS_scanned_One_colon:w
1628     } {
1629         \BNVS_scanned_Colons:w
1630     }
1631 }

```

❗ `\peek_regex_remove:NnTF` is buggy in L3 programming layer <2025-01-18>.

```

1632 \cs_new:Npn \BNVS_scan_colon:Fw #1 {
1633     \peek_regex_replace_once:NnTF \c__bnvs_Colons_regex { \cB\{ \1 \cE\} } {
1634         \BNVS_scanned_Colons_regex:n
1635     } {
1636         #1
1637     }
1638 }

```

Comma branch

\c__bnvs_comMa_regex Comma as a separator.

```
1639 \regex_const:Nn \c__bnvs_comMa_regex { \s* , \s* }
```

(End of definition for \c__bnvs_comMa_regex.)

\BNVS_scan_comMa:Fw	\BNVS_scan_comMa:Fw {<else:w>} <input stream>
\BNVS_scanned_comMa:w	\BNVS_scan_comMa:w <input stream>
\BNVS_scanned_comMa_set:n	\BNVS_scan_comMa_set:n {<comMa:w>}

Once a comma has just been scanned from the head of the <input stream>, calls \BNVS_scanned_comMa:w branch function, otherwise execute <else:w> instructions. \BNVS_scan_comMa_set:n sets the meaning of \BNVS_scan_comMa:w to <comMa:w>.

```
1640 \cs_new:Npn \BNVS_scan_comMa:Fw #1 {
  ❗ \peek_regex_remove:NnTF is buggy in L3 programming layer <2025-01-18>.
1641   \peek_regex_replace_once:NnTF \c__bnvs_comMa_regex {} {
1642     \BNVS_scanned_comMa:w
1643   } {
1644     #1
1645   }
```

❗ There must be absolutely no code here. No hooking allowed either.

```
1646 }
```

Close branch

\c__bnvs_cLose_regex Closing delimiters.

```
1647 \regex_const:Nn \c__bnvs_cLose_regex { \s* (%([{
1648   \}|\]||\)
1649   ) \s*
1650 }
```

(End of definition for \c__bnvs_cLose_regex.)

\BNVS_scan_cLose:Fw	\BNVS_scan_cLose:Fw {<else:w>} <input stream>
\BNVS_scanned_cLose:nw	\BNVS_scanned_cLose:nw <clositer> <input stream>
\BNVS_scanned_cLose:w	\BNVS_scanned_cLose:w <input stream>
\BNVS_scanned_cLose_set:n	\BNVS_scanned_cLose_set:n <cLose:w>

When the \BNVS_scan_cLose:Fw function scans one of ‘)’, ‘]’ or ‘}’ closing delimiter, it calls the branch function \BNVS_scanned_cLose:nw with that delimiter as argument, which in turn calls \BNVS_scanned_cLose:w, after it manages eventual errors in balancing delimiters. Otherwise execute <else:w> code.

```
1651 \cs_new:Npn \BNVS_scan_cLose:Fw #1 {
1652   \peek_regex_replace_once:NnTF \c__bnvs_cLose_regex { \cB\{ \1 \cE\} } {
1653     \BNVS_scanned_cLose:nw
1654   } {
1655     #1
1656   }
1657 }
```

```

1658 \cs_new:Npn \BNVS_scanned_cClose:nw #1 {
1659   \BNVS_error:n { Unexpected~delimiter~`#1' }
1660   \BNVS_scanned_cClose:w
1661 }

1662 \tl_map_inline:nn {{Block}{One_colon}{Colons}{comMa}{cClose}} {
1663   \cs_new:cpn { BNVS_scanned_#1_set:n } ##1 {
1664     \cs_set:cpn { BNVS_scanned_#1:w } {
1665       ##1
1666     }
1667   }

```

❗ Do nothing operation at the top level.

```

1668   \cs_new:cpn { BNVS_scanned_#1:w } {
1669   }
1670 }

```

Stop branch The Stop branch is executed in the Stop situation, *id est* when the head of the $\langle input\ stream \rangle$ cannot be scanned. In practice, when the head of the $\langle input\ stream \rangle$ is not a character of category code space or other.

<code>\BNVS_scan_Stop:F</code>	<code>\BNVS_scan_Stop:F {$\langle else:n \rangle$} $\langle input\ stream \rangle$</code>
<code>\BNVS_scanned_Stop:</code>	<code>\BNVS_scanned_Stop:</code>
<code>\BNVS_scanned_Stop_set:n</code>	<code>\BNVS_scanned_Stop_set:n {$\langle Stop: \rangle$}</code>

When the head of the $\langle input\ stream \rangle$ is not a space nor a character of category code other, `\BNVS_scan_Stop:F` calls the `\BNVS_scanned_Stop:` branch function, otherwise execute $\langle else:n \rangle$ instructions. The function `\BNVS_scanned_Stop_set:n` sets the meaning of `\BNVS_scanned_Stop:` to $\langle Stop: \rangle$.

```

1671 \cs_new:Npn \BNVS_scanned_Stop_set:n #1 {
1672   \cs_set:Npn \BNVS_scanned_Stop: {
1673     #1
1674   }
1675 }

❗ Do nothing operation at the top level.

1676 \cs_new:Npn \BNVS_scanned_Stop: {
1677 }

1678 \cs_new:Npn \BNVS_scan_Stop:F #1 {
1679   \peek_catcode:NTF \c_catcode_other_token { #1 } {
1680     \peek_catcode_remove:NTF \c_space_token {

1681       \BNVS_xprt_space:
1682       #1
1683     } {
1684       \BNVS_scanned_Stop:
1685     }
1686   }

```

⊘ There must be absolutely no code here. No hooking allowed either.

```

1687 }

```

1.12.5 Begin and end of scanning processes

`\BNVS_begin_scan:` `\BNVS_begin_scan:`

Start a scanning group. *Implementation detail:* the function is split into 3 parts for debugging and testing purposes.

```

1688 \cs_new:Npn \BNVS_begin_scan: {
1689   \BNVS_scan_will_begin:
1690   \BNVS_begin:
1691   \BNVS_scan_did_begin:
1692 }

1693 \cs_new:Npn \BNVS_scan_will_begin: {
1694   \BNVS_xprt_ast_Raw:
1695 }

1696 \cs_new:Npn \BNVS_scan_did_begin: {
1697   \BNVS_xprt_clear:
1698   \BNVS_ast_plc_wrap:n { ##1 }
1699   \BNVS_ast_prefix_set:n {}
1700   \BNVS_next_set:n {}
1701 }
```

`\BNVS_end_scan:n` `\BNVS_end_scan:n {⟨content⟩*}`
`\BNVS_end_scan:` `\BNVS_end_scan:`
`\BNVS_end_scan_no_I:` `\BNVS_end_scan_no_I:`
`\BNVS_end_scan_next:` `\BNVS_end_scan_next:`

Exactly one of these functions must be called to close the T_EX group opened while beginning the scanning process. `\BNVS_end_scan_next:` calls `\BNVS_end_scan:` and what was stored as next instructions before closing the group. `\BNVS_end_scan:` call `\BNVS_end_scan:n` on what was exported before closing the group (at this level), once prepared by `\BNVS_ast_plc:n...` `\BNVS_end_scan_no_I:` does not export I and S(*I*) variables.

```

1702 \cs_new:Npn \BNVS_end_scan:n #1 {
1703   \__bnvs_tl_if_empty:cTF { I } {
1704     \exp_args:NNx \BNVS_end: \BNVS_xprt_ast:n {
1705       \BNVS_ast_plc:n { #1 }
1706     }
1707   } {
1708     \BNVS_tl_use:nvv {
1709       \exp_args:Nx \BNVS_end_scan:nnn {
1710         \BNVS_ast_plc:n { #1 }
1711       }
1712     } { I } { S ( \l__bnvs_I_tl ) }
1713   }
1714 }

1715 \cs_new:Npn \BNVS_end_scan:nnn #1 #2 #3 {
1716   \BNVS_end:
1717   \BNVS_xprt_ast:n { #1 }
1718   \__bnvs_tl_set:cn I { #2 }
1719   \__bnvs_tl_set:cn { S ( #2 ) } { #3 }
1720 }
```



```

1721 \cs_new:Npn \BNVS_end_scan: {
1722   \BNVS_xprted_use:n {
1723     \BNVS_end_scan:n
1724   }
1725 }

1726 \cs_new:Npn \BNVS_end_scan_no_I: {
1727   \__bnvs_tl_clear:c I
1728   \BNVS_end_scan:
1729 }

1730 \cs_new:Npn \BNVS_end_scan_next: {
1731   \BNVS_next_use:n { \BNVS_end_scan: }
1732 }

```

1.12.6 Main scan functions

```

\BNVS_scan:w \BNVS_scan:w <input stream>
\BNVS_scan:nw \BNVS_scan:nw {<raw>} <input stream>

```

Medley of the above `\BNVS_scan_...:Fw` scan functions. Mutability is allowed by a prior call to `\BNVS_scan_Block_def:n`.

```

1733 \cs_new:Npn \BNVS_scan:w {
1734   \BNVS_scan_cLose:Fw {
1735     \BNVS_scan_colon:Fw {
1736       \BNVS_scan_Stop:F {
1737         \BNVS_scan_Block:Fw {
1738           \BNVS_scan_comMa:Fw {
1739             \BNVS_scan:nw
1740           }
1741         }
1742       }
1743     }
1744   }
1745 }

1746 \cs_new:Npn \BNVS_scan:nw #1 {
1747   \BNVS_xprt_Raw:n { #1 }
1748   \BNVS_scan:w
1749 }

```

```
\BNVS_scan:EBOCMLSTN:w \BNVS_scan:EBOCMLSTN:w {<prEamble:>}
    {<Block:w>} {<One_colon:w>} {<Colons:w>} {<comMa:w>} {<cLose:w>} {<Stop:>}
    {<posTamble:>} {<scAn:w>}
<input stream>
```

Start a scanning process by executing the `<prEamble:>` instructions. The six next arguments are function bodies used to define the corresponding branch functions. The `<posTamble:>` and `<scAn:w>` instructions are finally executed. They are expected to scan the input stream and terminate by calling one of the branch functions just defined. They consist of some `\BNVS_scan_...:...w` function defined below.

The scanning functions have instructions as argument. Instead of simply using `n` in the signature of such functions, we use letters `EBOCMLSTN` as defined here, with a supplemental colon separator. Using `b` in the signature means that the instructions will be followed by a call to the `\BNVS_scanned_Block:w` function.

```
1750 \cs_new:Npn \BNVS_scan:EBOCMLSTN:w #1 #2 #3 #4 #5 #6 #7 #8 #9 {
1751   #1
1752   \BNVS_scanned_Block_set:n { #2 }
1753   \BNVS_scanned_One_colon_set:n { #3 }
1754   \BNVS_scanned_Colons_set:n { #4 }
1755   \BNVS_scanned_comMa_set:n { #5 }
1756   \BNVS_scanned_cLose_set:n { #6 }
1757   \BNVS_scanned_Stop_set:n { #7 }
1758   #8
1759   #9
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
1760 }
```

```
\BNVS_begin_scan:nn \BNVS_begin_scan:nn {<xprt:n>} {<next:>}
```

⚠ **OBSOLETE.** Call `\BNVS_begin_scan:` with default arguments that end the scan group and call the eponym branch function.

```
1761 \cs_new:Npn \BNVS_begin_scan:nn #1 #2 {
1762   \BNVS_begin_scan:
1763   \BNVS_ast_plc_wrap:n { #1 }
1764   \BNVS_next_set:n { #2 }
1765 }
```

```
\BNVS_scan:ETN:w \BNVS_scan:ETN:w {<prEamble:>} {<posTamble:>} {<scAn_BOCLMS:w>} <input stream>
```

Start a scanning branch section by executing the `<prEamble:>` instructions and setting the branch functions to default values. The `<posTamble:>` instructions are then executed to eventually override these defaults. Finally, the `<scAn_BOCLMS:w>` instructions are executed. They must terminate with a call to one of the branch functions.

```
1766 \cs_new:Npn \BNVS_scan:ETN:w #1 #2 #3 {
1767   \BNVS_scan:EBOCMLSTN:w {
1768     \BNVS_begin_scan:
1769     #1
1770   } {
1771     \BNVS_end_scan_next: \BNVS_scanned_Block:w
1772   }
```

```

1773   \BNVS_end_scan_next: \BNVS_scanned_One_colon:w
1774 } {
1775   \BNVS_end_scan_next: \BNVS_scanned_Colons:w
1776 } {
1777   \BNVS_end_scan_next: \BNVS_scanned_comMa:w
1778 } {
1779   \BNVS_end_scan_next: \BNVS_scanned_cLose:w
1780 } {
1781   \BNVS_end_scan_next: \BNVS_scanned_Stop:
1782 } {
1783   #2
1784 } {
1785   #3
1786 }
1787 }

```

1.12.7 $\langle Rnd \rangle$, $\langle Sqr \rangle$ and $\langle Cr1 \rangle$ units

We assume that delimiters are paired and balanced. Nevertheless, some errors are caught and properly managed. Here is a grammar for material enclosed between delimiters

```

1  $\langle Rnd(\langle blnc \rangle) \rangle ::= \langle Rnd\_open \rangle \langle blnc \rangle \langle Rnd\_close \rangle$ 
2  $\langle Rnd\_open \rangle ::= \langle spc \rangle '(' \langle spc \rangle$ 
3  $\langle Rnd\_close \rangle ::= \langle spc \rangle ')' \langle spc \rangle$ 
4  $\langle Sqr(\langle blnc \rangle) \rangle ::= \langle Sqr\_open \rangle \langle blnc \rangle \langle Sqr\_close \rangle$ 
5  $\langle Sqr\_open \rangle ::= \langle spc \rangle '[' \langle spc \rangle$ 
6  $\langle Sqr\_close \rangle ::= \langle spc \rangle ']' \langle spc \rangle$ 
7  $\langle Cr1(\langle blnc \rangle) \rangle ::= \langle Cr1\_open \rangle \langle blnc \rangle \langle Cr1\_close \rangle$ 
8  $\langle Cr1\_open \rangle ::= \langle spc \rangle '{' \langle spc \rangle$ 
9  $\langle Cr1\_close \rangle ::= \langle spc \rangle '}' \langle spc \rangle$ 

```

with different variations of associate AST's. (missing description for $\langle Sqr \rangle$ and $\langle Cr1 \rangle$ are similar)

```

 $\langle Rnd(\langle blnc \rangle) \rangle^* ::= \langle prefix:w \rangle \{ \langle blnc \rangle^* \}$ 
|  $\langle prefix:w \rangle \langle blnc \rangle^*$ 
|  $\{ \langle blnc \rangle^* \}$ 
|  $\langle blnc \rangle^*$ 

```

The exported AST also depends on the exportation policy, whether $\langle blnc \rangle$ is empty or not.

```

\BNVS_scan_cLose:N:EbsTN:w \BNVS_scan_cLose:N:EbsTN:w <cLose>
\BNVS_scan_cLose:N:PXbsTN:w {<prEamble:>}& {<block:>}& {<stop:>}& {<posTamble:>}& {<scAn:w>}&
\BNVS_scan_cLose:N:PXbsTN:w <cLose>
{<xprt:n>}& {<prefix:w>}& {<block:>}& {<stop:>}& {<posTamble:>}& {<scAn:w>}&

```

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using the `<scAn:w>` instructions. Implementation detail: we begin a scanning group that will be closed on scanning the `<close>` delimiter, or a different closing delimiter, allways at the same level. The `<posTamble:>` allows some fine tuning. The `<scAn:w>` function scans the `<input stream>`, and is expected to end on a closing delimiter at the current level by sending `\BNVS_scanned_cLose:w`, which meaning is `<close:>` followed by `\BNVS_scanned_cLose:w`. If the `<scAn:w>` will call itself after a block, a comma or a colon if the corresponding branch function is called. This is the normal instruction flow.

```

1788 \cs_new:Npn \BNVS_scan_cLose:N:EbsTN:w #1 #2 #3 #4 #5 #6 {
1789   \BNVS_scan:EBOCMLSTN:w {

```

¶ One group level for the contents.

```

1790   \BNVS_begin_scan:
1791   \cs_set:Npn \BNVS_scanned_cLose:nw ##1 {

```

¶ Anyways, branch here.

```

1792       \BNVS_scanned_cLose:w
1793   }
1794   #2
1795 } {

```

¶ Any branch function, but the two last ones, executes the `<scAn:w>` instructions.

```

1796   #6
1797 } {
1798   #6
1799 } {
1800   #6
1801 } {
1802   #6
1803 } {

```

¶ Close branch. End the group, execute `<block:>` instructions and calls the function `\BNVS_scanned_Block:w`. Intermediate groups should be closed appropriately with the right definition of branch functions. The value of I is not exported to the containing group.

```

1804   \BNVS_end_scan_no_I:
1805   #3
1806   \BNVS_scanned_Block:w
1807 } {

```

¶ Stop branch. Execute the `<stop:>` instructions after the group ends.

```

1808   \BNVS_error:n { Missing~`#1'. }
1809   \BNVS_end_scan_no_I:
1810   #4
1811   \BNVS_scanned_Stop:
1812 } {

```

¶ Execute the `<posTamble:>` instructions.

```

1813     #5
1814 } {
1815     #6
1816 }
1817 }

1818 \cs_new:Npn \BNVS_scan_cLose:N:PXbsTN:w #1 #2 #3 {
1819     \BNVS_scan_cLose:N:EbsTN:w #1 {
1820         \BNVS_ast_plc_wrap_pfx:n { #2 }
1821         \BNVS_ast_prefix_set:n { #3 }
1822     }
1823 }

```

\c__bnvs_Rnd_open_regex Round opening delimiter.

```

1824 \regex_const:Nn \c__bnvs_Rnd_open_regex { \s* \( %)
1825     \s* }

```

(End of definition for \c__bnvs_Rnd_open_regex.)

\BNVS_scan_Rnd:EbsTN:Fw **\BNVS_scan_Rnd:EbsTN:Fw** {<prEamble:>} {<block:>} {<stop:>} {<posTamble:>}
{<scAn:w>} {<F:w>} <input stream>

Scan a <Rnd> unit if any and execute one of <block:> or <Stop:> instructions, otherwise execute <F:w> instructions.

```

1826 \cs_new:Npn \BNVS_scan_Rnd:EbsTN:Fw #1 #2 #3 #4 #5 #6 {
1827     \peek_regex_replace_once:NnTF \c__bnvs_Rnd_open_regex { } {
1828         \BNVS_scan_cLose:N:EbsTN:w %(
1829     ) {
1830         \BNVS_ast_plc_wrap_grp:n { ##1 }
1831         #1
1832     } { #2 } { #3 } { #4 } { #5 }
1833 } {
1834     #6
1835 }
1836 }

```

\c__bnvs_Sqr_open_regex Square opening delimiter.

```

1837 \regex_const:Nn \c__bnvs_Sqr_open_regex { \s* \[ %)
1838     \s* }

```

(End of definition for \c__bnvs_Sqr_open_regex.)

\BNVS_scan_Sqr:EbsTN:Fw **\BNVS_scan_Sqr:EbsTN:Fw** {<prEamble:>} {<block:>} {<stop:>} {<posTamble:>}
{<scAn:w>} {<F:w>} <input stream>

Scan <Sqr> unit with the help of <scAn:w>, then execute one of \BNVS_scanned_block: or \BNVS_scanned_Stop: instructions. <scAn:w> must end with one of the branch functions \BNVS_scanned_cLose:w or \BNVS_scanned_Stop:. Otherwise execute <F:w> instructions.

```

1839 \cs_new:Npn \BNVS_scan_Sqr:EbsTN:Fw #1 #2 #3 #4 #5 #6 {
1840   \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex { } {
1841     \BNVS_scan_cClose:N:EbsTN:w %[
1842       ] { #1 } { #2 } { #3 } { #4 } { #5 }
1843   } {
1844     #6
1845   }
1846 }

```

`\c__bnvs_Cr1_open_regex` Curly opening delimiter.

(End of definition for `\c__bnvs_Cr1_open_regex`.)

```

1847 \regex_const:Nn \c__bnvs_Cr1_open_regex { \s* \{ %}
1848   \s* }

```

`\BNVS_scan_Cr1:EbsTN:Fw` `\BNVS_scan_Cr1:EbsTN:Fw {⟨prEamble:⟩} {⟨block:⟩} {⟨stop:⟩} {⟨scAn:w⟩} {⟨F:w⟩}`
 ⟨input stream⟩

Scan `⟨Cr1⟩` unit with the help of `⟨scAn:w⟩`, then execute one of `⟨block:⟩` or `⟨Stop:⟩` instructions. `⟨scAn:w⟩` must end with one of the branch functions `\BNVS_scanned_cClose:w` or `\BNVS_scanned_Stop:.` Otherwise execute `⟨F:w⟩` instructions.

```

1849 \group_begin:
1850 \char_set_catcode_group_begin:N (
1851 \char_set_catcode_group_end:N )
1852 \char_set_catcode_other:N \{
1853 \char_set_catcode_other:N \}
1854 \cs_new:Npn \BNVS_scan_Cr1:EbsTN:Fw #1 #2 #3 #4 #5 #6 (
1855   \peek_regex_replace_once:NnTF \c__bnvs_Cr1_open_regex ( ) (
1856     \BNVS_scan_cClose:N:EbsTN:w %[
1857       ] ( #1 ) ( #2 ) ( #3 ) ( #4 ) ( #5 )
1858   ) (
1859     #6
1860   )
1861 )
1862 \group_end:

```

1.12.8 Raw units

```

5  ⟨Raw⟩      ::= ⟨Raw_open⟩ ⟨Call⟩ ⟨Rnd_close⟩
6  ⟨Raw_open⟩ ::= ⟨spc⟩ ' "( ' ⟨spc⟩
7  ⟨RawLst⟩   ::= ( ⟨RawRnd⟩ | ⟨RawSqr⟩ | ⟨RawCr1⟩ | ⟨RawOtr⟩ ) *
8  ⟨RawRnd⟩   ::= ⟨Rnd_open⟩ ⟨RawLst⟩ ⟨Rnd_close⟩
9  ⟨RawSqr⟩   ::= ⟨Sqr_open⟩ ⟨RawLst⟩ ⟨Sqr_close⟩
10 ⟨RawCr1⟩   ::= ⟨Cr1_open⟩ ⟨RawLst⟩ ⟨Cr1_close⟩
11 ⟨RawOtr⟩   ::= ⟨characters of category other except delimiters⟩

```

`\c__bnvs_open_Raw_regex` Raw opening delimiter.

```

1863 \regex_const:Nn \c__bnvs_open_Raw_regex { \s* "\ ( %}
1864   \s* }

```

(End of definition for \c__bnvs_open_Raw_regex.)

\BNVS_scan_Raw_LS:w	\BNVS_scan_Raw_LS:w <input stream>
\BNVS_scan_Raw:nw	\BNVS_scan_Raw:nw {<input stream>} <input stream>

Scan balanced material taking only delimiters into account. It stops on an unbalanced closing delimiter or in a stop situation. Everything in between is exported as is.

```

1865 \cs_new:Npn \BNVS_scan_Raw:nw #1 {
1866   \BNVS_xprt_ast:n { #1 }
1867   \BNVS_scan_Raw_LS:w
1868 }
```

```

1869 \group_begin:
1870 \char_set_catcode_group_begin:N <
1871 \char_set_catcode_group_end:N >
1872 \char_set_catcode_other:N \{
1873 \char_set_catcode_other:N \}
1874 \cs_new:Npn \BNVS_scan_Raw_LS:w <
```

❗ If we find an opening delimiter.

```

1875   \BNVS_scan_Rnd:EbsTN:Fw <
1876   \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >% (
1877   \BNVS_ast_plc_wrap_mpt:n < ##1 ) >
1878   > <
1879   \BNVS_scanned_cLose_set:n <
1880   \BNVS_end_scan_no_I:
1881   \BNVS_scan_Raw_LS:w
1882   >
1883   > <% (
1884   \BNVS_error:n < Missing~`)' > % (
1885   \BNVS_xprt_ast:n < ) >
1886   > <
1887   \BNVS_xprt_ast:n < ( > %)
1888   > <
1889   \BNVS_scan_Raw_LS:w
1890   > <
1891   \BNVS_scan_Sqr:EbsTN:Fw <
1892   \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >% [
1893   \BNVS_ast_plc_wrap_mpt:n < ##1 ] >
1894   > <
1895   \BNVS_scanned_cLose_set:n <
1896   \BNVS_end_scan_no_I:
1897   \BNVS_scan_Raw_LS:w
1898   >
1899   > <% [
1900   \BNVS_error:n < Missing~`]' > %[
1901   \BNVS_xprt_ast:n < ] >
1902   > <
1903   \BNVS_xprt_ast:n < [ > %]
1904   > <
1905   \BNVS_scan_Raw_LS:w
1906   > <
1907   \BNVS_scan_Cr1:EbsTN:Fw <
```

```

1908      \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >{%{
1909      \BNVS_ast_plc_wrap_mpt:n < ##1 } >
1910    > <
1911      \BNVS_scanned_cLose_set:n <
1912      \BNVS_end_scan_no_I:
1913      \BNVS_scan_Raw_LS:w
1914    >
1915    > <%{
1916      \BNVS_error:n < Missing~`}' > {%{
1917      \BNVS_xprt_ast:n < } >
1918    > <
1919      \BNVS_xprt_ast:n < { > }%{
1920    > <
1921      \BNVS_scan_Raw_LS:w
1922    > <

```

¶ The head of the $\langle input\ stream \rangle$ is not an opening delimiter, we look for a closing delimiter, or a stop situation or finally export one character.

```

1923      \BNVS_scan_cLose:Fw < \BNVS_scan_Stop:F < \BNVS_scan_Raw:nw > >
1924    > > >
1925    \group_end:

```

$\backslash\text{BNVS_scan_Raw:bs:Fw}$ $\backslash\text{BNVS_scan_Raw:bs:Fw}$ $\{ \langle block: \rangle \} \{ \langle stop: \rangle \} \{ \langle F:w \rangle \} \langle input\ stream \rangle$

Scan balanced material taking only delimiters into account. Starts on a "(".

```

1926 \cs_new:Npn \BNVS_scan_Raw:bs:Fw #1 #2 #3 {
1927   \peek_regex_replace_once:NnTF \c__bnvs_open_Raw_regex { } {
1928     \BNVS_scan_cLose:N:EbsTN:w %(
1929   ) {
1930     \BNVS_ast_plc_wrap_pfx:n { \:n { ##1 } }
1931     \BNVS_xprt_space_set:n { \BNVS_xprt_ast:n { ~ } }
1932   } { #1 } { #2 } { } {
1933     \BNVS_scan_Raw_LS:w
1934   }
1935 } {
1936   #3
1937 }
1938 }

```

1.12.9 $\langle csl(\langle item \rangle) \rangle$ units

Here is a partial grammar for a comma separated list template.

$$\langle csl(\langle item \rangle) \rangle ::= \langle item \rangle? (\langle comma \rangle \langle item \rangle?)^*$$

The corresponding AST template is below, where $\langle item \rangle^*$ is the AST for $\langle item \rangle$:

$$\langle csl(\langle item \rangle) \rangle^* ::= \{ \langle item \rangle^* \} \{ \langle item \rangle^* \} \langle \dots \rangle \{ \langle item \rangle^* \}$$

```
\BNVS_scan_csl:ElsTN:w \BNVS_scan_csl:ElsTN:w {⟨prEamble:⟩} {⟨close:⟩} {⟨stop:⟩} {⟨posTamble:⟩}
{⟨scAn:w⟩} ⟨input stream⟩
```

This function scans the `⟨input stream⟩` for a comma separated list of items, regardless enclosing delimiters, according to the `⟨scAn:w⟩` instructions. These instruction must end with a branch function, of which only `\BNVS_scanned_cLose:w` and `\BNVS_scanned_Stop:` are caught. The scanned material is then exported according to the instructions given in the `⟨prEamble:⟩`. On scanning a comma, a new group is created. The exported material is empty for an empty list, for input `a` it is `{a*}`, for input `a,b` it is `{a*}{b*}`, and so on.

```
1939 \cs_new:Npn \BNVS_scan_csl:ElsTN:w #1 #2 #3 #4 #5 {
1940   \BNVS_scan:EBOCMLSTN:w {
❧ One group level for each item of the list.
1941     \BNVS_begin_scan:
❧ Execute the ⟨prEamble:⟩ instructions.
1942     \BNVS_ast_plc_wrap:n { { ##1 } }
1943     #1
1944   }
❧ For the first brach functions, execute the ⟨scAn:w⟩ instructions.
1945     { #5 }
1946     { #5 }
1947     { #5 }
1948     {
1949       \BNVS_xprted_use:n { \tl_if_empty:nTF } {
❧ Comma branch, the first item is empty, skip it.
1950       #5
1951     } {
❧ Comma branch, export the first item when not empty. Forwards I, to the outer group.
1952       \BNVS_end_scan:
1953       \BNVS_scan_csl:ElsTN:w { #1 } { } { } { #4 } { #5 }
1954     }
1955   }
❧ Close branch.
1956   { \BNVS_end_scan: #2 \BNVS_scanned_cLose:w }
1957   { \BNVS_end_scan: #3 \BNVS_scanned_Stop: }
❧ By default, the list contains only one item.
1958   { #4 }
1959   { #5 }
1960 }
```

```
\BNVS_scan_csl:ET:w \BNVS_scan_csl:ET:w {⟨prEamble:⟩} {⟨posTamble:⟩} ⟨input stream⟩
```

These functions scan the comma separated list of items, regardless the enclosing delimiters, based on `\BNVS_scan:w`.

```

1961 \cs_new:Npn \BNVS_scan_csl:ET:w #1 #2 {
1962   \BNVS_scan_csl:ElsTN:w
1963   { #1 }
1964   { }
1965   { }
1966   { #2 }
1967   { \BNVS_scan:w }
1968 }

```

1.12.10 ISR unit

Here is a partial grammar for identifiers,

```

2  <ISR>      ::= ( <IS> | <dot_P> ) <R>*
3  <IS>       ::= ( <I>? '!' )? <S>
4  <I>        ::= <S>
5  <S>        ::= <alpha> <alnum>*
6  <dot_P>    ::= '.' <P>
7  <P>        ::= <digit>* <S>
8  <R>        ::= <dot_P> | <dot_N> | <n>
9  <dot_N>    ::= '.' <N>
10 <n>        ::= <Sqr_open> <n_expr> <Sqr_close>
11 <n_expr>   ::= <blnc>

```

and the associate AST :

```

<IS>* ::= \IS:w { <I> } { <S> }
<ISR>* ::= \ISR:w { <I> } { <S> } { <R>* }
<R>* ::= ( <P>* | <N>* | <n>* ) *
<P>* ::= { \P:n { <P> } }
<N>* ::= { \N:n { <N> } }
<n>* ::= { \n:n { <n_expr>* } }

```

\c__bnvs_IKS_regex The start of a qualified dotted name, with three capture groups.

(End of definition for \c__bnvs_IKS_regex.)

```

1969 \regex_const:Nn \c__bnvs_IKS_regex {
    1: the optional frame <id> with
    2: the optional exclamation mark
1970   (? : ( \ur{c__bnvs_S_regex} )? ( ! ) )?
    3: followed by a non empty short name.
1971   ( \ur{c__bnvs_S_regex} )
1972 }

```

```
\BNVS_scanned::B:IKS:w \BNVS_scanned::B:IKS:w {<scAn::B:w>} {<I>} {<K>} {<S>} <input stream>
```

Only called by `\BNVS_scan_IS::B:Fw`. Execute `<scAn::B:w>` instructions, which is the body of a function with one argument allways used last in the workflow. This argument is replaced by `\BNVS_scanned_Block:w`.

```
1973 \cs_new:Npn \BNVS_scanned::B:IKS:w #1 #2 #3 #4 {
1974   \tl_if_empty:nTF { #3 } {
```

❗ No frame identifier given, use the current value. Store the short name, store in the appropriate variable.

```
1975     \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
1976     \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #4 }
1977   } {
```

❗ Frame identifier given. Proceed as before mutatis mutandis.

```
1978     \__bnvs_tl_set:cn I { #2 }
1979     \__bnvs_tl_clear_new:c { S(#2) }
1980     \__bnvs_tl_set:cn { S(#2) } { #4 }
1981   }
1982   \__bnvs_tl_set:cn S { #4 }
```

❗ Create a scan group to manage the `<prefix:w>`.

```
1983   \BNVS_scan:ETN:w {
```

❗ By default we start on a standalone identifier,

```
1984     \BNVS_ast_plc_wrap:n { ##1 }
1985     \BNVS_ast_prefix_set:n { \IS:w }
1986   } {
```

❗ Export `<I>` and `<S>`.

```
1987     \BNVS_tl_use:Nv \BNVS_xprt_grp:n I
1988     \BNVS_xprt_grp:n { #4 }
1989   } {
```

❗ Execute the `<scAn::B:w>` instructions including the terminating branch function `\BNVS_scanned_Block:w`. The `<prefix:w>` will possibly change.

```
1990     \cs_set:Npn \BNVS_scAn:B:w ##1 { #1 }
1991     \BNVS_scAn:B:w { \BNVS_scanned_Block:w }
1992   }
1993 }
```

```
\BNVS_scan_IS::B:Fw \BNVS_scan_IS::B:Fw {<scAn::B:w>} {<F:w>} <input stream>
```

Scan an `<IS>` unit if any, stores the `<I>`, `<S>` between braces, and calls `\BNVS_scanned::B:IKS:w` that will execute `<scAn::B:w>` instructions. The `<F:w>` instructions are executed when no `<IS>` unit is found.

```
1994 \cs_new:Npn \BNVS_scan_IS::B:Fw #1 #2 {
1995   \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex {
1996     \cB\{ \1 \cE\} \cB\{ \2 \cE\} \cB\{ \3 \cE\}
1997   } {
1998     \BNVS_scanned::B:IKS:w { #1 }
1999   } {
```

❗ Choose the false branch.

```

2000     #2
2001   }
2002 }

```

\BNVS_scanning_PNn:P:w **\BNVS_scanning_PNn:P:w** {*P*} *<input stream>*

Export the *<P>* component then scan another *<PNn>* component.

```

2003 \cs_new:Npn \BNVS_scanning_PNn:P:w #1 {
2004   \BNVS_xprt_grp:n { \P:n { #1 } }
2005   \BNVS_scan_PNn:w
2006 }

```

\BNVS_scanning_PNn:N:w **\BNVS_scanning_PNn:P:w** {*N*} *<input stream>*

Export the *<N>* component then scan another *<PNn>* component.

```

2007 \cs_new:Npn \BNVS_scanning_PNn:N:w #1 {
❧ Normalize <N>.
2008   \exp_args:Ne \BNVS_xprt_grp:n { \exp_not:N \N:n { \int_eval:n { #1 } } }
2009   \BNVS_scan_PNn:w
2010 }

```

\BNVS_scanning_PNn_n:w **\BNVS_scanning_PNn_n:w** *<input stream>*

Only called by **\BNVS_scan_PNn:w**.

```

2011 \cs_new:Npn \BNVS_scanning_PNn_n:w {
2012   \BNVS_scan_cLose:N:EbsTN:w %[
2013   ] {
2014     \BNVS_ast_plc_wrap:n { { \n:n { ##1 } } }
2015     { } { } { } {
2016
❧ On cLose branch, scan <PNn> after <n>.
2017     \BNVS_scanned_cLose_set:n { \BNVS_end_scan_no_I: \BNVS_scan_PNn:w }
2018     { {
2019       \BNVS_scan_n_expr_LS:w
2020     } }
2021 }

```

\BNVS_scan_PNn:w **\BNVS_scan_PNn:w** *<input stream>*

Scan all the individual *<R>* components, exporting them one after the other, then execute **\BNVS_scanned_Block:w** branch function. Calls **\BNVS_scanning_PNn:P:w**, **\BNVS_scanning_PNn:N:w** or **\BNVS_scanning_PNn_n:w**.

```

2022 \cs_new:Npn \BNVS_scan_PNn:w {
2023   \peek_charcode_remove:NTF . {
2024     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
2025       \BNVS_scanning_PNn:P:w
2026     } {
2027       \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
2028         \BNVS_scanning_PNn:N:w

```

```
2029     } {
```

❗ Terminates the block and reinject the dot ‘.’.

```
2030     \BNVS_scanned_Block:w .
```

```
2031     }
```

```
2032     }
```

```
2033     } {
```

```
2034     \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex { } {
```

```
2035     \BNVS_scanning_PNn:n:w
```

```
2036     } {
```

❗ Also terminates the block.

```
2037     \BNVS_scanned_Block:w
```

```
2038     }
```

```
2039     }
```

```
2040 }
```

```
\BNVS_scanning_R:B:P:w \BNVS_scanning_R:B:P:w { \<scan_B:w> } { \<P> } \<input stream>
```

Only called by `\BNVS_scan_R:B:w` when a heading `<P>` has just been scanned. Calls `\BNVS_scan_PNn:w`.

```
2041 \cs_new:Npn \BNVS_scanning_R:B:P:w #1 #2 {
```

❗ New scan group to gather the `<R>` relative path.

```
2042     \BNVS_scan:ETN:w {
```

```
2043         \BNVS_ast_plc_wrap_grp:n { ##1 }
```

```
2044         \BNVS_xprt_grp:n { \P:n { #2 } }
```

```
2045     } {
```

```
2046         \BNVS_scanned_Block_set:n {
```

```
2047             \BNVS_end_scan_no_I:
```

```
2048             #1
```

```
2049         }
```

```
2050     } {
```

```
2051         \BNVS_scan_PNn:w
```

```
2052     }
```

```
2053 }
```

```
\BNVS_scanning_R:B:N:w \BNVS_scanning_R:B:N:w { \<scan_B:w> } { \<N> } \<input stream>
```

Only called by `\BNVS_scan_R:B:w` when a heading `<N>` has just been scanned. Calls `\BNVS_scan_PNn:w`.

```
2054 \cs_new:Npn \BNVS_scanning_R:B:N:w #1 #2 {
```

❗ New scan group to gather the `<R>` relative path.

```
2055     \BNVS_scan:ETN:w {
```

```
2056         \BNVS_ast_plc_wrap_grp:n { ##1 }
```

```
2057         \exp_args:Ne \BNVS_xprt_grp:n {
```

❗ Normalize `<N>`.

```
2058             \exp_not:N \N:n { \int_eval:n { #2 } }
```

```
2059         }
```

```
2060     } {
```

```
2061         \BNVS_scanned_Block_set:n {
```

```

2062     \BNVS_end_scan_no_I:
2063     #1
2064 }
2065 } {
2066     \BNVS_scan_PNn:w
2067 }
2068 }

```

\BNVS_scanning_R_n:B:w **\BNVS_scanning_R_n:B:w** {*<scAn_B:w>*} *<input stream>*

Only called by **\BNVS_scan_R:B:w** when a heading '[' has just been scanned. Terminates by **\BNVS_scan_PNn:w** to scan another component. See above for *<scAn::B:w>*.

```

2069 \cs_new:Npn \BNVS_scanning_R_n:B:w #1 {

```

¶ New scan group to gather the *<R>* relative path.

```

2070     \BNVS_scan:ETN:w {
2071         \BNVS_ast_plc_wrap_grp:n { ##1 }
2072     } {

```

¶ On block branch, close the scan group and execute the *<scAn_B:w>* instructions.

```

2073         \BNVS_scanned_Block_set:n {
2074             \BNVS_end_scan_no_I:
2075             #1
2076         }
2077     } {

```

¶ Export material up to the closing square bracket as one single argument group.

```

2078         \BNVS_scan_cLose:N:EbsTN:w %[
2079         ] {
2080             \BNVS_ast_plc_wrap:n { { \n:n { ##1 } } }
2081         } { } { } { } {

```

¶ On close branch, scan *<PNn>* after *<n>*.

```

2082         \BNVS_scanned_cLose_set:n { \BNVS_end_scan_no_I: \BNVS_scan_PNn:w }
2083     } {
2084         \BNVS_scan_n_expr_LS:w
2085     }
2086 }
2087 }

```

\BNVS_scan_R:B:w **\BNVS_scan_R:B:w** {*<scAn_B:w>*} *<input stream>*

Scan a *<R>* component and execute *<scAn_B:w>* instructions. The scanning stops exactly when some regular expression matches, or when squared material is scanned. The function **\BNVS_scan_R:B:w** calls

- **\BNVS_scanning_R:B:P:w**,
- **\BNVS_scanning_R:B:N:w** or
- **\BNVS_scanning_R_n:B:w**

with *<scAn_B:w>* as argument.

```

2088 \cs_new:Npn \BNVS_scan_R:B:w #1 {

```

```

2089 \peek_charcode_remove:NnTF . {
2090   \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
2091     \BNVS_scanning_R:B:P:w { #1 }
2092   } {
2093     \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
2094       \BNVS_scanning_R:B:N:w { #1 }
2095     } {

```

❗ Terminates the block and reinject the dot.

```

2096       \BNVS_xprt_grp:n { }
2097       #1
2098       .
2099     }
2100   }
2101 } {
2102   \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex { } {
2103     \BNVS_scanning_R_n:B:w { #1 }
2104   } {

```

❗ Terminates the block too.

```

2105       \BNVS_xprt_grp:n { }
2106       #1
2107     }
2108   }
2109 }

```

\BNVS_scanning_R::B:w \BNVS_scanning_R::B:w {<scan::B:w>} <input stream>

Only called by \BNVS_scan_R::B:w when the head of the <input stream> has nothing special .

```

2110 \cs_new:Npn \BNVS_scanning_R::B:w #1 {
2111   \cs_set:Npn \BNVS:w {
2112     \cs_set:Npn \BNVS_scan:B:w #####1 { #1 }
2113   }
2114   \BNVS:w
2115   \BNVS_scan:B:w { \BNVS_scanned_Block:w }
2116 }
2117 % \end{KWN.macrocode}
2118 % ^^A
2119 % \begin{function}{
2120 %   \BNVS_scanning_R::B:P:w,
2121 % }
2122 % \begin{syntax}
2123 % \KWNlatex*{\BNVS_scanning_R::B:P:w}^^A
2124 % \KWNlatex*{\BNVS_scan:B:w}^^A
2125 % \KWNlatex*{\BNVS_scan_PNn:w}^^A
2126 % \KWNlatex*{\BNVS_input_stream}
2127 % \end{syntax}
2128 % Only called by
2129 % \KWNlatex[to:N=\BNVS_scanning_R::B:P:w]{\BNVS_scan_R::B:w}
2130 % when a heading \KWN\ has just been scanned.
2131 % Calls
2132 % \KWNlatex[from:N=\BNVS_scanning_R::B:N:w]{\BNVS_scan_PNn:w}.
2133 % \end{function}

```

```

2134 % \begin{KWN.macrocode}
2135 % \begin{macrocode}
2136 \cs_new:Npn \BNVS_scanning_R::B:P:w #1 #2 {
  ⓘ New scan group to gather the  $\langle R \rangle$  relative path.

2137   \BNVS_scan:ETN:w {
2138     \BNVS_ast_plc_wrap_grp:n { ##1 }
2139     \BNVS_xprt_grp:n { \P:n { #2 } }
2140   } {
2141     \BNVS_scanned_Block_wrap::B { #1 }
2142   } {
2143     \BNVS_scan_PNn:w
2144   }
2145 }

```

\BNVS_scanning_R::B:N:w **\BNVS_scanning_R::B:N:w** { $\langle block: \rangle$ } { $\langle N \rangle$ } $\langle input\ stream \rangle$

Only called by **\BNVS_scan_R::B:w** when a heading $\langle N \rangle$ has just been scanned. Calls **\BNVS_scan_PNn:w**.

```

2146 \cs_new:Npn \BNVS_scanning_R::B:N:w #1 #2 {
  ⓘ New scan group to gather the  $\langle R \rangle$  relative path.

2147   \BNVS_scan:ETN:w {
2148     \BNVS_ast_plc_wrap_grp:n { ##1 }
2149     \exp_args:Ne \BNVS_xprt_grp:n {

  ⓘ Normalize  $\langle N \rangle$ .

2150     \exp_not:N \N:n { \int_eval:n { #2 } }
2151   }
2152   } {
2153     \BNVS_scanned_Block_wrap::B { #1 }
2154   } {
2155     \BNVS_scan_PNn:w
2156   }
2157 }

```

\BNVS_scanning_R_n::B:w **\BNVS_scanning_R_n::B:w** { $\langle scAn::B:w \rangle$ } $\langle input\ stream \rangle$

Only called by **\BNVS_scan_R::B:w** when a heading '[' has just been scanned. Terminates by **\BNVS_scan_PNn:w** to scan another component. See above for $\langle scAn::B:w \rangle$.

```

2158 \cs_new:Npn \BNVS_scanning_R_n::B:w #1 {
  ⓘ New scan group to gather the  $\langle R \rangle$  relative path.

2159   \BNVS_scan:ETN:w {
2160     \BNVS_ast_plc_wrap_grp:n { ##1 }
2161   } {

  ⓘ On block branch, execute the  $\langle scAn:w \rangle$  instructions.

2162     \BNVS_scanned_Block_wrap::B { #1 }
2163   } {

  ⓘ Export material up to the closing square bracket as one single argument group.

```



```

2164 \BNVS_scan_cLose:N:EbsTN:w %[
2165 ] {
2166     \BNVS_ast_plc_wrap:n { { \n:n { ##1 } } }
2167 } { } { } { }

```

¶ On close branch, scan $\langle Pn \rangle$ after $\langle n \rangle$.

```

2168     \BNVS_scanned_cLose_set:n { \BNVS_end_scan_no_I: \BNVS_scan_PNn:w }
2169 } {
2170     \BNVS_scan_n_expr_LS:w
2171 }
2172 }
2173 }

```

\BNVS_scan_R::B:w \BNVS_scan_R::B:w { $\langle scAn::B:w \rangle$ } $\langle input\ stream \rangle$

Scan a $\langle R \rangle$ component and execute $\langle scAn:w \rangle$ instructions. The scanning stops exactly when some regular expression matches, or when squared material is scanned. The function \BNVS_scan_R::B:w calls

- \BNVS_scanning_R::B:P:w,
- \BNVS_scanning_R::B:N:w or
- \BNVS_scanning_R_n::B:w

with $\langle scAn::B:w \rangle$ as argument.

```

2174 \cs_new:Npn \BNVS_scan_R::B:w #1 {
2175     \peek_charcode_remove:NTF . {
2176         \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
2177             \BNVS_scanning_R::B:P:w { #1 }
2178         } {
2179             \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
2180                 \BNVS_scanning_R::B:N:w { #1 }
2181             } {

```

¶ Terminates the block and reinject the dot.

```

2182         \BNVS_xprt_grp:n { }
2183         \BNVS_scanning_R::B:w { #1 }
2184     .
2185 }
2186 }
2187 } {
2188     \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex { } {
2189         \BNVS_scanning_R_n::B:w { #1 }
2190     } {

```

¶ Terminates the block too.

```

2191     \BNVS_xprt_grp:n { }
2192     \cs_set:Npn \BNVS:w {
2193         \cs_set:Npn \BNVS_scan:B:w #####1 { #1 }
2194     }
2195     \BNVS:w
2196     \BNVS_scan:B:w { \BNVS_scanned_Block:w }

```

```

2197     }
2198   }
2199 }

```



FIASKO ‘ $\{\{\backslash n:n\{4+1\}\}\backslash P:n\{P2P\}\}\backslash n:n\{6+6\}\}\backslash N:n\{421\}\}\backslash n:n\{a+b=x\}\}\}!=‘\{\{\backslash 4+1\}\}\backslash P:n\{P2P\}\}\backslash n:n\{6+6\}\}\backslash P:n\{421\}\}\backslash n:n\{a+b=x\}\}\}’$ (beanoves.dtx:15894)



Z

```

2200 \tl_new:N \l__bnvs_other_X_tl
2201 \tl_set:Nx \l__bnvs_other_X_tl { \tl_to_str:n { X } }

```

\BNVS_scanned_dot:AP:w **\BNVS_scanned_dot:AP:w** $\{\langle scAn:B:w\rangle\}\{\langle P\rangle\}$ $\langle input\ stream\rangle$

Only called by **\BNVS_scan_dot_P:A:Fw**.

```

2202 \cs_new:Npn \BNVS_scanned_dot:AP:w #1 #2 {

```

No frame identifier given, no short name given.

```

2203   __bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {

```

It defaults to X.

```

2204     \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
2205     __bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
2206   }
2207   __bnvs_tl_set:cv S { S( \BNVS_tl_use:c I ) }

```

Create a scan group to manage the $\langle prefix:w\rangle$. It will be closed by a call to **\BNVS_scanned_Block:w**.

```

2208   \BNVS_scan:ETN:w {

```

By default we start on a standalone identifier,

```

2209     \BNVS_ast_prefix_set:n { \ISR:w }
2210     \exp_args:Nx \BNVS_ast_plc_wrap_mpt:n {
2211       { __bnvs_tl_use:c I }
2212       { __bnvs_tl_use:c S }
2213       { \exp_not:n { ##1 } }
2214     }
2215     \BNVS_xprt_ast:n { { \P:n { #2 } } }
2216   } {

```

Create a scan group to manage the $\langle prefix:w\rangle$.

```

2217     \BNVS_scanned_Block_wrap::n { #1 }
2218   } {

```

Find the next components.

```

2219     \BNVS_scan_PNn:w
2220   }
2221 }

```

\BNVS_scan_dot_P:A:Fw **\BNVS_scan_dot_P:A:Fw** $\{\langle scAn:B:w\rangle\}\{\langle F:w\rangle\}$ $\langle input\ stream\rangle$

Scan a $\langle dot_P\rangle$ unit and execute $\langle scAn:B:w\rangle$ instructions that must terminate with the branch function **\BNVS_scanned_Block:w**. The $\langle F:w\rangle$ instructions are executed when no $\langle dot_P\rangle$ unit is found.

```

2222 \cs_new:Npn \BNVS_scan_dot_P:A:Fw #1 #2 {
2223   \peek_charcode_remove:NTF . {
2224     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
2225       \BNVS_scanned_dot:AP:w { #1 }
2226     } {

```

❗ Choose the false branch and reinject the dot.

```

2227       #2
2228     .
2229   }
2230 } {

```

❗ Choose also the false branch.

```

2231   #2
2232 }
2233 }

```

\BNVS_scanned_dot:AN:w \BNVS_scanned_dot:AN:w {<scan:B:w>} {<N>} <input stream>

Only called by \BNVS_scan_dot_N:A:Fw.

```

2234 \cs_new:Npn \BNVS_scanned_dot:AN:w #1 #2 {

```

❗ No frame identifier given, no short name given.

```

2235   \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {

```

It defaults to X.

```

2236     \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
2237     \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
2238   }
2239   \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I ) }

```

❗ Create a scan group to manage the <prefix:w>. It will be closed by a call to \BNVS_scanned_Block:w.

```

2240   \BNVS_scan:ETN:w {

```

❗ By default we start on a standalone identifier,

```

2241     \BNVS_ast_prefix_set:n { \ISR:w }
2242     \exp_args:Ne \BNVS_ast_plc_wrap_mpt:n {
2243       { \__bnvs_tl_use:c I }
2244       { \__bnvs_tl_use:c S }
2245       { \exp_not:n { ##1 } }
2246     }
2247     \exp_args:Ne \BNVS_xprt_ast:n {
2248       { \exp_not:N \N:n { \int_eval:n { #2 } } }
2249     }
2250   } {

```

❗ Create a scan group to manage the <prefix:w>.

```

2251     \BNVS_scanned_Block_wrap:n { #1 }
2252   } {

```

❗ Find the next components.

```

2253     \BNVS_scan_PNn:w
2254 }
2255 }

```

```

\BNVS_scan_dot_N:A:Fw \BNVS_scan_dot_N:A:Fw {<scAn:B:w>} {<F:w>} <input stream>

```

Scan a $\langle \text{dot}_N \rangle$ unit and execute $\langle \text{scAn:B:w} \rangle$ instructions that must end with $\backslash \text{BNVS_scanned_Block:w}$. The $\langle \text{F:w} \rangle$ instructions are executed when no $\langle \text{dot}_N \rangle$ unit is found.

```

2256 \cs_new:Npn \BNVS_scan_dot_N:A:Fw #1 #2 {
2257   \peek_charcode_remove:NTF . {
2258     \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
2259       \BNVS_scanned_dot:AN:w { #1 }
2260     } {

```

❗ Choose the false branch and reinject the dot.

```

2261       #2
2262     .
2263   }
2264 } {

```

❗ Choose also the false branch.

```

2265     #2
2266   }
2267 }

```

```

\BNVS_scan_ISR::B:Fw \BNVS_scan_ISR::B:Fw {<scAn:b:>} {<F:w>} <input stream>

```

Scan an $\langle \text{ISR} \rangle$ unit if any, $\langle \text{scAn:b:>} \rangle$ instructions followed by a call to the branch function $\backslash \text{BNVS_scanned_Block:w}$. The $\langle \text{F:w} \rangle$ instructions are executed when no $\langle \text{ISR} \rangle$ unit is found.

```

2268 \cs_new:Npn \BNVS_scan_ISR::B:Fw #1 #2 {
2269   \BNVS_scan_IS::B:Fw {
2270     \BNVS_scan_R::B:w { #1 }
2271   } {
2272     #2
2273   }
2274 }

```

1.12.11 $\langle \text{MOR} \rangle$, calls and assignment units

Here is the grammar

```

7  <ISRCMOR> ::= <ISR> | <ISRC> | <ISRMOR>
8  <ISRMOR>  ::= <ISR> <MOR>
9  <MOR>     ::= <ModOpEq> <Rhs>
10 <ModOpEq> ::= <spc> ( <Mod> <Op> '=' ) <spc>
11 <Mod>     ::= ' ' | '?'
12 <Op>      ::= ' ' | '+' | '-' | '/' | '*'
13 <Rhs>     ::= <see below>

```

with the associate AST

$$\langle ISRMOR \rangle^* ::= \backslash ISRMOR:w \{ \langle I \rangle \} \{ \langle S \rangle \} \{ \langle R \rangle^* \} \{ \langle Mod \rangle \} \{ \langle Op \rangle \} \{ \langle Rhs \rangle^* \}$$

$$\hookrightarrow \langle Rhs \rangle^*$$

`\c__bnvs_ModOpEq_regex` Assignment operators, two capture groups.

(End of definition for `\c__bnvs_ModOpEq_regex`.)

```
2275 \regex_const:Nn \c__bnvs_ModOpEq_regex {
2276   \s*

      1: an optional modifier.
2277   ( \? )?

      2: an optional operator.
2278   ( [-+/*] )?

2279   =\s*
2280 }
```

`\BNVS_scanned_ModOpEq:MLS:nnw` `\BNVS_scanned_ModOpEq:MLS:nnw { \<comMa:w> } { \<cLose:w> } { \<Stop:> } { \<Mod> } { \<Op> } \<input stream>`

Only called by `\BNVS_scan_MOR:MLS:Fw`. Calls `\BNVS_scan_MLS:w`.

```
2281 \cs_new:Npn \BNVS_scanned_ModOpEq:MLS:nnw #1 #2 #3 #4 #5 {
2282   \BNVS_xprt_grp:n { #4 }
2283   \BNVS_xprt_grp:n { #5 }
2284   \BNVS_scan:EBOCMLSTN:w {
2285     \BNVS_ast_plc_wrap_grp:n { ##1 }
2286   } { \BNVS_error:n { Unexpected } \BNVS_scanned_cLose:w }
2287   { \BNVS_error:n { Unexpected } \BNVS_scanned_cLose:w }
2288   { \BNVS_error:n { Unexpected } \BNVS_scanned_cLose:w }
2289   { \BNVS_end_scan: #1 }
2290   { \BNVS_end_scan: #2 }
2291   { \BNVS_end_scan: #3 }
2292   { }
2293   { \BNVS_scan_MLS:w }
2294 }
```

`\BNVS_scan_MOR:MLS:Fw` `\BNVS_scan_MOR:MLS:Fw { \<comMa:w> } { \<cLose:w> } { \<Stop:> } { \<F:w> } \<input stream>`

Scan a `\<ModOpEq>` unit, if any, and execute one of `\<comMa:w>`, `\<cLose:w>` or `\<Stop:>` instructions, otherwise execute `\<F:w>` instructions.

```
2295 \cs_new:Npn \BNVS_scan_MOR:MLS:Fw #1 #2 #3 #4 {
2296   \peek_regex_replace_once:NnTF \c__bnvs_ModOpEq_regex {
2297     \cB\{ \1 \cE\} \cB\{ \2 \cE\}
2298   } {
2299     \BNVS_scanned_ModOpEq:MLS:nnw { #1 } { #2 } { #3 }
2300   } {
2301     #4
2302   }
2303 }
```